UCD-SeRG Lab Manual

Last updated: 2026-10-12

Contents

1 Welcome to UCD-SeRG! 1
- 1.1 About the lab . . . 1
- 1.2 About this lab manual . . . 1

2 Culture and conduct 2
- 2.1 Lab culture . . . 2
- 2.2 Diversity, equity, and inclusion . . . 2
- 2.3 Protecting human subjects . . . 2
- 2.4 Authorship . . . 3

3 Communication and coordination 4
- 3.1 Microsoft Teams . . . 4
- 3.2 Email . . . 4
- 3.3 Direct Hyperlinks in Internal Correspondence . . . 5
- 3.4 Managing Notifications and Work-Life Balance . . . 5
- 3.5 Setting Away Status and Autoresponse Checklist . . . 5
- 3.6 Task Management . . . 6
- 3.7 Shared Documents . . . 6
- 3.8 UC Davis Box and SharePoint . . . 6
- 3.9 Meetings . . . 7
- 3.10 Conference Room Booking . . . 8
- 3.11 Code Review . . . 8

4 Reproducibility 9
- 4.1 What is the reproducibility crisis? . . . 9
- 4.2 Study design . . . 9
- 4.3 Register study protocols . . . 10
- 4.4 Write and register pre-analysis plans . . . 10
- 4.5 Create reproducible workflows . . . 10
- 4.6 Process and analyze data with internal replication and masking . . . 11
- 4.7 Use reporting checklists with manuscripts . . . 11
- 4.8 Publish preprints . . . 11
- 4.9 Publish data (when possible) and replication scripts . . . 11

5 Code repositories 12

6 R Coding Practices 13

7 R Code Style 15

8 Big data 17

9 Data masking 18

10 Quarto 19

11 Github **20**
11.1 GitHub Education and Copilot Access . . . 21

12 Collaborative Software Development **23**

13 Continuous Integration **24**

14 Unix **25**

15 Reproducible Environments **26**

16 Code Publication **27**

17 Data Publication **28**
17.1 Overview . . . 28
17.2 Removing PHI . . . 29
17.3 Create public IDs . . . 30
17.4 Create a data repository . . . 31
17.5 Edit and test analysis scripts . . . 32
17.6 Create a public GitHub page for public scripts . . . 32
17.7 Go live . . . 33

18 High-performance computing (HPC) **34**
18.1 UC Davis Computing Resources . . . 34
18.2 Getting started with SLURM clusters . . . 34
18.3 Moving files to the cluster . . . 36
18.4 Installing packages on the cluster . . . 36
18.5 Testing your code . . . 37
18.6 Running graphical programs with X11 forwarding . . . 39
18.7 Recovering a disconnected interactive session . . . 42
18.8 Storage & group storage access . . . 43
18.9 Running big jobs . . . 44

19 Working with AI **46**
19.1 Institutional AI account access . . . 46

20 Checklists **48**
20.1 Onboarding checklist . . . 48
20.2 Pre-analysis plan checklist . . . 48
20.3 Code checklist . . . 49
20.4 Manuscript checklist . . . 49
20.5 Figure checklist . . . 50
20.6 Offboarding checklist . . . 50

21 Resources **51**
21.1 Resources for R . . . 51
21.2 Professional advice . . . 51
21.3 Funding . . . 51
21.4 Ethics and global health research . . . 52
21.5 Comparable Lab Manuals . . . 52
21.6 Reading and Watching List . . . 52

22 Professional Development **53**
22.1 Mentoring Philosophy . . . 53
22.2 Individual Development Plans . . . 53
22.3 Navigating the Hidden Curriculum . . . 53
22.4 Presentations and Conferences . . . 55
22.5 Grant Writing . . . 55
22.6 PhD Dissertation Requirements . . . 55
22.7 Teaching and Outreach . . . 57
22.8 Networking . . . 58

23 Writing **59**
23.1 Writing to Clarify Your Thinking . . . 59
23.2 Plain, Direct Prose . . . 59
23.3 Scanning for AI Tells . . . 60
23.4 Show, Don't Tell . . . 60

24 Manuscript Preparation and Publication **62**
24.1 Publication Process . . . 62
24.2 Responding to Peer Review . . . 62
24.3 Preprints and Open Access . . . 63
24.4 Reporting Checklists . . . 63
24.5 Manuscript Checklist . . . 64
24.6 Formatting tables with `gtsummary` + `flextable` + `flexlsx` . . . 64
24.7 Scientific Writing: Claims and Evidence . . . 65

References **66**

Appendices **68**

Copilot Instructions File **68**

Copilot Setup Steps File **69**

Document Generation Metadata **70**

1 Welcome to UCD-SeRG!

1.1 About the lab

Welcome to the UC Davis Seroepidemiology Research Group (SeRG), led by Drs. Kristen Aiemjoy and Ezra Morrison. Accurate methods to measure infectious disease burden are essential for guiding public health decisions, yet many infectious diseases remain underrecognized due to limited diagnostics and costly, resource-intensive surveillance systems. Our work addresses this gap by developing seroepidemiologic methods to characterize infection burden in populations. Currently, we focus on enteric fever (Salmonella Typhi and Paratyphi), Scrub Typhus (Orientia tsutsugamushi), Melioidosis (Burkholderia pseudomallei), Shigella (Shigella spp.), and Cholera (Vibrio cholerae). We are supported by the US National Institutes of Health, the Bill and Melinda Gates Foundation, and the Department of Defense, and collaborate with partners around the world. To learn more about the lab, visit ucdserg.ucdavis.edu[1].

1.2 About this lab manual

This lab manual covers our communication strategy, code of conduct, and best practices for reproducibility of computational workflows. It is a living document that is updated regularly.

This manual is a fork of the Benjamin-Chung Lab Manual (Benjamin-Chung et al. 2024), adapted for UCD-SeRG. We are grateful to Dr. Jade Benjamin-Chung and her team for developing and openly sharing their excellent lab manual. You can view the original manual at jadebc.github.io/lab-manual[2]. Original contributors include Jade Benjamin-Chung, Kunal Mishra, Stephanie Djajadi, Nolan Pokpongkiat, Anna Nguyen, Iris Tong, and Gabby Barratt Heitmann. The Data Publication chapter also draws from Fanice Nyatigo and Ben Arnold's public-data chapter in the Proctor Lab at UCSF handbook[3].

Feel free to draw from this manual (and please cite it if you do!).

This work is licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("Creative Commons Attribution-NonCommercial 4.0 International License," n.d.).

[1] https://ucdserg.ucdavis.edu
[2] https://jadebc.github.io/lab-manual/index.html
[3] https://proctor-ucsf.github.io/dcc-handbook/publicdata.html

2 Culture and conduct

2.1 Lab culture

We are committed to a lab culture that is collaborative, supportive, inclusive, open, and free from discrimination and harassment.

We encourage students / staff of all experience levels to respectfully share their honest opinions and ideas on any topic. Our group has thrived upon such respectful honest input from team members over the years, and this document is a product of years of student and staff input (and even debate) that has gradually improved our productivity and overall quality of our work.

2.2 Diversity, equity, and inclusion

UCD-SeRG recognizes the importance of and is committed to cultivating a culture of diversity, equity, and inclusion. This means being a safe, supportive, and anti-racist environment in which students from diverse backgrounds are equally and inclusively supported in their education and training. Diversity takes many forms, and includes, but is not limited to, differences in race, ethnicity, gender, sexuality, socioeconomic status, religion, disability, and political affiliation.

2.3 Protecting human subjects

All lab members must complete CITI Human Subjects Biomedical Group 1[1] training and share their certificate with the lab leadership. Team members will be added to relevant Institutional Review Board protocols prior to their start date to ensure they have permission to work with identifiable datasets.

One of the most relevant aspects of protecting human subjects in our work is maintaining confidentiality. For students supporting our data science efforts, in practice this means:

- Be sure to understand and comply with project-specific policies about where data can be saved, particularly if the data include personal identifiers.
- Do not share data with anyone without permission, including to other members of the group, who might not be on the same IRB protocol as you (check with lab leadership first).

Remember, data that looks like it does not contain identifiers to you might still be classified as data that requires special protection by our IRB or under HIPAA, so always proceed with caution and ask for help if you have any concerns about how to maintain study participant confidentiality.

[1] https://research.ucdavis.edu/policiescompliance/irb-admin/education/

2.4 Authorship

We adhere to the ICMJE Definition of authorship[2] (International Committee of Medical Journal Editors, n.d.) and are happy for team members who meet the definition of authorship to be included as co-authors on scientific manuscripts. To qualify for authorship, individuals must meet all four criteria:

1. Substantial contributions to conception/design, or acquisition/analysis/interpretation of data
2. Drafting the work or revising it critically for important intellectual content
3. Final approval of the version to be published
4. Agreement to be accountable for all aspects of the work

Authorship practices:

- **First authorship**: Typically goes to the person who led the work
- **Corresponding author**: Usually the PI, unless otherwise agreed
- **Co-authorship**: Determined by substantial intellectual contributions
- **Author order**: Should be discussed and agreed upon by all authors
- **Acknowledgments**: For contributions that don't meet authorship criteria

Authorship should be discussed early in a project and revisited as the work evolves to ensure transparency and fairness. We encourage using the CRediT Taxonomy[3] to document specific author contributions.

[2] http://www.icmje.org/recommendations/browse/roles-and-responsibilities/defining-the-role-of-authors-and-contributors.html
[3] https://credit.niso.org/

3 Communication and coordination

One benefit of the academic environment is its schedule flexibility and autonomy. This means that lab members may choose to work in the early morning, afternoon, evening, or weekends. Lab members should manage their own notification schedules to ensure they can maintain healthy work-life boundaries. We do not expect lab members to respond outside of normal business hours (unless there are special circumstances). Because of the nature of our work across time zones, frequent travel, and international collaborations, there are no set working hours. Team members are not expected to reply to messages outside their own working hours, even if a message arrives outside of those hours.

3.1 Microsoft Teams

- Use Microsoft Teams for scheduling, coding related questions, quick check ins, etc. If your Teams message exceeds 200 words, it might be time to use email.
- Use channels instead of direct messages unless you need to discuss something private.
- Please make an effort to respond to messages that mention you (e.g., `@username`) as quickly as possible.
- If you are unusually busy (e.g., preparing for QE/grant deadline, taking two exams) or on vacation please alert the team in advance so we can expect you not to respond at all / as quickly as usual. Please also set your status in Teams (e.g., it could say "On vacation") so we know not to expect to see you online.
- Please thread messages in Teams as much as possible.
- When referencing a pull request, issue, commit, or file, always include a direct hyperlink to where you want your teammates to look (see Direct Hyperlinks in Internal Correspondence[1]).
- Don't wait for meetings to ask questions. As soon as a question comes up, write it out in Teams. This benefits both you (by clarifying your thinking, as discussed in Chapter 23) and the team (by getting the conversation started earlier).

3.2 Email

- Use email for longer messages (>200 words) or messages that merit preservation.
- When discussing specific code, pull requests, or documents in emails, include direct hyperlinks so recipients do not have to search for the relevant items (see Direct Hyperlinks in Internal Correspondence[2]).
- Generally, strive to respond within a work day when you are directly addressed in an email. Again because we work across many timezones, this means responding within a work day in your own timezone. If you're on vacation, you're not expected to respond

[1] @sec-direct-hyperlinks
[2] @sec-direct-hyperlinks

until the end of your first work day back (or possibly longer, if you've been out for a while and have a backlog).
- As noted above, if you are unusually busy or on vacation please alert the team in advance and via an away message, so we can expect you not to respond at all / as quickly as usual.

3.3 Direct Hyperlinks in Internal Correspondence

In all internal correspondence—including Microsoft Teams messages, emails, GitHub issues, and pull request comments—always provide direct hyperlinks to the exact place you want your audience to look.

- **Link directly to the target:** Provide a direct, clickable URL or markdown link to the specific pull request, issue, commit, branch, file, or line number.
- **Do not make your audience search:** Avoid vague references (such as "I opened a PR", "see the latest changes on my branch", or "look at the helper script") or bare unlinked identifiers (such as `#123` in emails or chat). Your audience should never have to search repositories, branches, or directory trees to find what you are referring to.
- **Use descriptive markdown links:** Format links with descriptive anchor text (for example, `[PR #123: Add data validation](https://github.com/UCD-SERG/lab-manual/pull/123)` or `[cleaning-script.R lines 45-60](https://github.com/...)`) so collaborators know where the link leads before clicking.
- **Reduce friction for collaborators:** Direct links save everyone time, prevent confusion about which branch or version is being discussed, and allow reviewers to immediately jump into context with a single click.

3.4 Managing Notifications and Work-Life Balance

Lab members should actively manage their notification schedules, especially if they receive work email or Teams notifications on their phones.

- **Block notifications outside of work hours**: We recommend configuring your devices and applications to prevent work notifications during non-work time. We all need uninterrupted rest and relaxation time in order to stay healthy and productive in the medium and long term.
- **Sending messages outside work hours**: There's no expectation to only send emails or messages during normal work hours. Feel free to send messages whenever you compose them—don't spend effort scheduling messages for later delivery. Recipients will manage their own notification schedules, and they may be working different hours or want the information sooner.

3.5 Setting Away Status and Autoresponse Checklist

When you'll be away from work (vacation, conference, sick leave, etc.), please help the team stay coordinated by setting appropriate away indicators:

- ☐ **Outlook calendar**: Mark yourself as away on your Outlook calendar for the dates you'll be unavailable.

- ☐ **Autoresponse**: Set an autoresponse message that indicates when you expect to be back at work.
- ☐ **Teams status**: Update your Teams status to indicate you're away (e.g., "On vacation until [date]").

3.6 Task Management

We use a combination of tools to track and manage project tasks:

- **GitHub Issues and Projects**: For code-related tasks, feature requests, and bug tracking. Lab leadership will assign issues and organize them in GitHub Projects. Issues are prioritized within projects, and you can track your assigned tasks there.
- **Microsoft Teams & Planner**: For general lab tasks and personal task management. Lab leadership may assign tasks through these tools, which integrate with Microsoft Teams. Lab members are expected to set up a Teams page for important degree requirements (QE) and projects (grant proposals, papers, etc.). Invite both Dr Morrison and Aiemjoy to the Teams page and assign them to review tasks with the deadline you need the review by. We appreciate 2 weeks' notice for letters and larger review projects.
- Generally, strive to complete assigned tasks by the date listed.
- Use checklists to break down tasks into smaller chunks. Sometimes leadership will create these for you, but you can also add them yourself.
- Update task status as you make progress so the team can stay coordinated.

3.7 Shared Documents

- We mostly use Microsoft 365 to collaborate and write internally and GoogleDocs when working with external collaborators. These documents may be linked to in GitHub Issues or other task tracking tools. Lab leadership often shares these with the whole team since tasks are overlapping, and even if a task is assigned to one person, others may have valuable insights.

3.8 UC Davis Box and SharePoint

- Human subjects data for research studies are generally stored in UC Davis Box or SharePoint. Please check with lab leadership about whether there are special storage and transfer requirements for the datasets you are working with for each study.
- You can access Box via your UC Davis credentials. For more information, visit UC Davis Box Support[3].
- SharePoint is also used for collaborative document storage and team file sharing. Access SharePoint through your UC Davis Microsoft 365 account.

[3] https://servicehub.ucdavis.edu/servicehub?id=ucd_kb_article&sysparm_article=KB0000184

3.9 Meetings

- Our meetings start on the hour.
- If you are going to be late, please send a message in our Teams channel.
- If you are regularly not able to come on the hour, notify the team and we might choose to modify the agenda order or the start time.

3.9.1 Developing Meeting Agendas

An effective meeting agenda ensures that collaborative time is productive, focused, and respectful of everyone's schedule. Whether organizing a weekly lab meeting, a project synchronization check-in, or a one-on-one advising session, follow these guidelines when preparing agendas.

3.9.1.1 Why agendas are important

- **Clear purpose and focus**: Meetings without an agenda easily drift into unstructured conversation or unfocused status updates. A written agenda clarifies the meeting's objective before bringing people together.
- **Preparation and respect for time**: Circulating an agenda in advance allows participants to review relevant materials, gather required data, and arrive ready to contribute substantive insights.
- **Accountability and action items**: A structured agenda provides a natural framework for tracking decisions, assigning clear ownership for action items, and reviewing progress at subsequent meetings.

3.9.1.2 Core content: Inform, Discuss, Ask

Organize your agenda items using three distinct functional categories:

- **Inform**: Brief announcements, status summaries, or project milestone updates. These items share essential context that requires little to no debate. Whenever possible, share background details asynchronously before the meeting so live time can focus on deeper discussion.
- **Discuss**: Topics requiring collaborative problem-solving, methodological debate, troubleshooting, or synthesis of differing viewpoints. These items benefit most from synchronous conversation.
- **Ask**: Specific requests for decisions, approvals, feedback, or resources from advisors, PIs, or team members. Clearly framing what you need (for example, "Decision on primary analysis model" or "Approval to submit abstract") helps attendees provide decisive guidance.

3.9.1.3 Formatting and style guidelines

- **Use clear category tags**: Prefix each bullet point with its category tag (such as `[Inform]`, `[Discuss]`, or `[Ask]`) so participants immediately understand their expected role for each topic.

- **Include direct links**: Always include direct hyperlinks to relevant draft manuscripts, code repositories, pull requests, or data summaries rather than requiring attendees to search for materials.
- **Allocate time estimates**: Assign estimated time allocations to each major agenda topic (for example, `[Discuss] Simulation study edge cases (15 min)`). This prevents early topics from crowding out crucial decisions scheduled at the end.
- **Distribute in advance**: Share the agenda in the calendar invitation, by email, or in the relevant Microsoft Teams channel at least 24 hours before the meeting.
- **Record action items**: Conclude every meeting by summarizing agreed-upon next steps, assigned owners, and target completion dates.

3.10 Conference Room Booking

The MS1C Conference Room can be booked through its Outlook calendar.

To book the room:

1. Create a new event in Outlook Calendar
2. Add the conference room as an attendee by typing "ms1c"
3. Select "MS1C Conference Room" from the results
4. Send the meeting invitation

The room will automatically accept or decline based on its availability.

To view the room's calendar:

1. In Outlook Calendar, click "Add Calendar"
2. Select "Add Shared Calendar…" or use "Open Calendar…"
3. Search for "ms1c"
4. Select "MS1C Conference Room" to add it to your calendar view

This allows you to check the room's availability before scheduling.

3.11 Code Review

Code review guidance has moved to its own chapter: see Collaborative Software Development[4].

[4] https://morrison-lab.github.io/psc/chapters/collaborative-software-development.html

4 Reproducibility

Our lab adopts the following practices to maximize the reproducibility of our work.

1. Design studies with appropriate methodology and adherence to best practices in epidemiology and biostatistics
2. Register study protocols
3. Write and register pre-analysis plans
4. Create reproducible workflows
5. Process and analyze data with internal replication and masking
6. Use reporting checklists with manuscripts
7. Publish preprints
8. Publish data (when possible) and replication scripts

4.1 What is the reproducibility crisis?

In the past decade, an increasing number of studies have found that published study findings could not be reproduced. Researchers found that it was not possible to reproduce estimates from published studies: 1) with the same data and same or similar code and 2) with newly collected data using the same (or similar) study design. These "failures" of reproducibility were frequent enough and broad enough in scope, occurring across a range of disciplines (epidemiology, psychology, economics, and others) to be deeply troubling. Program and policy decisions based on erroneous research findings could lead to wasted resources, and at worst, could harm intended beneficiaries. This crisis has motivated new practices in reproducibility, transparency, and openness. Our lab is committed to adopting these best practices, and much of the remainder of the lab manual focuses on how to do so.

Recommended readings on the "reproducibility crisis":

- Nuzzo R. How scientists fool themselves – and how they can stop (Nuzzo 2015)
- Stoddart C. Is there a reproducibility crisis in science? (Stoddart 2019)
- Munafò MR, et al. A manifesto for reproducible science (Munafò et al. 2017)

4.2 Study design

Appropriate study design is beyond the scope of this lab manual and is something trainees develop through their coursework and mentoring.

4.3 Register study protocols

We register all randomized trials on clinicaltrials.gov, and in some cases register observational studies as well.

4.4 Write and register pre-analysis plans

We write pre-analysis plans for most original research projects that are not exploratory in nature, although in some cases, we write pre-analysis plans for exploratory studies as well. The format and content of pre-analysis plans can vary from project to project. Here is an example of one: https://osf.io/tgbxr/. Generally, these include:

1. Brief background on the study (a condensed version of the introduction section of the paper)
2. Hypotheses / objectives
3. Study design
4. Description of data
5. Definition of outcomes
6. Definition of interventions / exposures
7. Definition of covariates
8. Statistical power calculation
9. Statistical analysis:

- Type of model
- Covariate selection / screening
- Standard error estimation method
- Missing data analysis
- Assessment of effect modification / subgroup analyses
- Sensitivity analyses
- Negative control analyses

4.5 Create reproducible workflows

Reproducible workflows allow a user to reproduce study estimates and ideally figures and tables with a "single click". In practice, this typically means running a single bash script that sources all replication scripts in a repository. These replication scripts complete data processing, data analysis, and figure/table generation. The following chapters provide detailed guidance on this topic:

- Chapter 5: Code repositories
- Chapter 6: Coding practices
- Chapter 7: Coding style
- Chapter 8: Code publication
- Chapter 9: Working with big data
- Chapter 10: Github
- Chapter 11: Unix

For additional learning resources on reproducible research practices, see the UC Davis DataLab workshop on reproducible research[1].

4.6 Process and analyze data with internal replication and masking

See my video on this topic: https://www.youtube.com/watch?v=WoYkY9MkbRE

4.7 Use reporting checklists with manuscripts

Using reporting checklists helps ensure that peer-reviewed articles contain the information needed for readers to assess the validity of your work and/or attempt to reproduce it. A collection of reporting checklists is available from the EQUATOR Network[2] ("EQUATOR Network," n.d.). Commonly used checklists in our lab include:

- CONSORT[3] (Consolidated Standards of Reporting Trials) for randomized trials
- STROBE[4] (Strengthening the Reporting of Observational Studies in Epidemiology) for observational studies
- STARD[5] (Standards for Reporting Diagnostic Accuracy Studies) for diagnostic accuracy studies

Please ensure that your manuscript follows these reporting guidelines before sending it for review.

4.8 Publish preprints

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv ("medRxiv," n.d.).

4.9 Publish data (when possible) and replication scripts

Publishing data and replication scripts allows other scientists to reproduce your work and to build upon it. We typically publish data on the Open Science Framework ("Open Science Framework," n.d.), share links to Github[6] repositories, and archive code on Zenodo[7].

[1] https://github.com/ucdavisdatalab/workshop_reproducible_research
[2] https://www.equator-network.org/
[3] https://www.equator-network.org/reporting-guidelines/consort/
[4] https://www.equator-network.org/reporting-guidelines/strobe/
[5] https://www.equator-network.org/reporting-guidelines/stard/
[6] github.com
[7] zenodo.org

5 Code repositories

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Code repositories**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Template repositories[3]
- Package Structure[4]
- `.Rproj` files[5]
- Organizing the `data-raw` folder[6]
- Repository Status Badges[7]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/code-repositories.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/code-repositories.html
[3] https://morrison-lab.github.io/psc/chapters/code-repositories.html#sec-template-repositories
[4] https://morrison-lab.github.io/psc/chapters/code-repositories.html#sec-package-structure
[5] https://morrison-lab.github.io/psc/chapters/code-repositories.html#sec-rproj-files
[6] https://morrison-lab.github.io/psc/chapters/code-repositories.html#sec-organizing-the-data-raw-folder
[7] https://morrison-lab.github.io/psc/chapters/code-repositories.html#sec-repostatus

6 R Coding Practices

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **R Coding Practices**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Lab Protocols for Code and Data[3]
- The Tool-Building Mindset[4]
- Essential R Package Development Tools[5]
- Complete Package Development Workflow[6]
- Organizing scripts[7]
- Testing Requirements[8]
- Test-Driven Development[9]
- Benchmarking[10]
- Iterative Operations[11]
- Reading and Saving Data[12]
- Version Control and Collaboration[13]
- Quality Assurance Checklist[14]
- Automated Code Styling[15]
- Documenting your code[16]
- Design Documentation with ADRs[17]
- Object naming[18]
- Function calls[19]
- Function decomposition[20]
- One function per file[21]
- Function length limits[22]

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/coding-practices.html
[3] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-lab-protocols-for-code-and-data
[4] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-tool-building
[5] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-package-tools
[6] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-workflow
[7] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-organizing-scripts
[8] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-testing
[9] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-tdd
[10] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-benchmarking
[11] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-iteration
[12] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-data-io
[13] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-version-control
[14] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-qa-checklist
[15] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-auto-styling
[16] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-documenting-code
[17] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-adrs
[18] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-coding-practices-object-naming
[19] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-function-calls
[20] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-function-decomposition
[21] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-one-function-per-file
[22] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-function-length-limits

- Keep It Short and Simple (KISS)[23]
- Decomposing Code Chunks in Quarto Documents[24]
- The here package[25]
- Reading/Saving Data[26]
- Integrating Box and Dropbox[27]
- r-lib Packages[28]
- Tidyverse[29]
- Coding with R and Python[30]
- Repeating analyses with different variations[31]
- Reviewing Code[32]
- Getting Help with Code[33]
- Additional Resources[34]

Looking for a specific section that used to be here?

This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/coding-practices.html#sec-foo` on the psc site.

[23] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-kiss
[24] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-decompose-qmd-chunks
[25] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-here-package-practices
[26] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-readingsaving-data
[27] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-integrating-box-and-dropbox
[28] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-lib-packages
[29] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-tidyverse
[30] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-coding-with-r-and-python
[31] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-repeating-analyses-with-different-variations
[32] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-reviewing-code
[33] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-getting-help-with-code
[34] https://morrison-lab.github.io/psc/chapters/coding-practices.html#sec-r-resources

7 R Code Style

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **R Code Style**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- General Principles[3]
- Clean Code Principles[4]
- Function Structure and Documentation[5]
- Assertions and Input Validation[6]
- Avoiding Deep Nesting[7]
- Prefer Existing Packaged Functions[8]
- Prefer Per-Operation Grouping[9]
- Avoid Hard-Coding Data with an External Source of Truth[10]
- Construct Complex Inputs Before the Call[11]
- Comments[12]
- Line Breaks and Formatting[13]
- Markdown and Quarto Formatting[14]
- Messaging and User Communication[15]
- Package Code Practices[16]
- Tidyverse Replacements[17]
- The here Package[18]
- Object Naming[19]
- Automated Tools for Style and Project Workflow[20]
- Additional Resources[21]

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/coding-style.html
[3] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-general-principles
[4] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-clean-code
[5] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-function-docs
[6] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-assertions-input-validation
[7] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-avoid-nesting
[8] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-prefer-packaged-functions
[9] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-per-operation-grouping
[10] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-avoid-hardcoding-external-data
[11] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-construct-inputs
[12] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-comments
[13] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-line-breaks-and-formatting
[14] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-markdown-and-quarto-formatting
[15] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-messaging-and-user-communication
[16] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-package-code-practices
[17] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-tidyverse-replacements
[18] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-here-package-style
[19] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-object-naming
[20] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-style-auto-style
[21] https://morrison-lab.github.io/psc/chapters/coding-style.html#sec-r-resources-style

Looking for a specific section that used to be here?

This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/coding-style.html#sec-foo` on the psc site.

8 Big data

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Big data**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- The data.table package[3]
- Using downsampled data[4]
- Optimal RStudio set up[5]

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/big-data.html
[3] https://morrison-lab.github.io/psc/chapters/big-data.html#sec-the-data-table-package
[4] https://morrison-lab.github.io/psc/chapters/big-data.html#sec-using-downsampled-data
[5] https://morrison-lab.github.io/psc/chapters/big-data.html#sec-optimal-rstudio-set-up

9 Data masking

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Data masking**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- General Overview[3]
- Data masking vs. tidy selection[4]
- Technical Overview[5]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/data-masking.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/data-masking.html
[3] https://morrison-lab.github.io/psc/chapters/data-masking.html#sec-general-overview
[4] https://morrison-lab.github.io/psc/chapters/data-masking.html#sec-data-masking-vs-tidy-selection
[5] https://morrison-lab.github.io/psc/chapters/data-masking.html#sec-technical-overview

10 Quarto

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Quarto**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Introduction[3]
- Quarto Basics[4]
- Building Quarto Books[5]
- Dashboards[6]
- Quarto Profiles[7]
- Advanced Features[8]
- Mermaid Diagrams[9]
- Troubleshooting[10]
- Additional Resources[11]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, usually under the same anchor. For example, `#sec-foo` here is `chapters/quarto.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/quarto.html
[3] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-quarto-introduction
[4] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-quarto-basics
[5] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-building-quarto-books
[6] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-quarto-dashboards
[7] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-quarto-profiles
[8] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-advanced-features
[9] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-mermaid-diagrams
[10] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-troubleshooting
[11] https://morrison-lab.github.io/psc/chapters/quarto.html#sec-quarto-additional-resources

11 Github

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Git and GitHub**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Basics[3]
- Graphical Git Clients[4]
- GitHub CLI and GitLab CLI[5]
- Git Branching[6]
- Personal Dev Branches[7]
- Example Workflow[8]
- Pull Request Roles and Protocol[9]
- Commonly Used Git Commands[10]
- How often should I commit?[11]
- Repeated Amend Workflow[12]
- What should be pushed to Github?[13]
- GitLab[14]
- Customizing How Files Appear on GitHub[15]
- Exporting Issue and Pull Request Conversations[16]
- Removing Sensitive Data from a Repository[17]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/github.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/github.html
[3] https://morrison-lab.github.io/psc/chapters/github.html#sec-github-basics
[4] https://morrison-lab.github.io/psc/chapters/github.html#sec-git-gui-clients
[5] https://morrison-lab.github.io/psc/chapters/github.html#sec-cli-tools
[6] https://morrison-lab.github.io/psc/chapters/github.html#sec-git-branching
[7] https://morrison-lab.github.io/psc/chapters/github.html#sec-dev-branches
[8] https://morrison-lab.github.io/psc/chapters/github.html#sec-example-workflow
[9] https://morrison-lab.github.io/psc/chapters/github.html#sec-pull-request-roles-and-protocol
[10] https://morrison-lab.github.io/psc/chapters/github.html#sec-commonly-used-git-commands
[11] https://morrison-lab.github.io/psc/chapters/github.html#sec-how-often-should-i-commit
[12] https://morrison-lab.github.io/psc/chapters/github.html#sec-repeated-amend-workflow
[13] https://morrison-lab.github.io/psc/chapters/github.html#sec-what-should-be-pushed-to-github
[14] https://morrison-lab.github.io/psc/chapters/github.html#sec-gitlab
[15] https://morrison-lab.github.io/psc/chapters/github.html#sec-gitattributes
[16] https://morrison-lab.github.io/psc/chapters/github.html#sec-export-conversations
[17] https://morrison-lab.github.io/psc/chapters/github.html#sec-removing-sensitive-data

11.1 GitHub Education and Copilot Access

11.1.1 GitHub Education Benefits

Students, faculty, and researchers at UC Davis can access GitHub Education benefits, which include free access to GitHub Pro features and various developer tools.

To sign up for GitHub Education:

1. Visit the GitHub Education website[18]
2. Click "Get benefits" or "Join GitHub Education"
3. Sign in with your GitHub account (or create one if you don't have one)
4. Complete the application form using your UC Davis email address (ending in `ucdavis.edu`)
5. Provide proof of your academic affiliation (e.g., upload a photo of your student ID or university letter)
6. Wait for approval (typically takes a few days)

Once approved, you'll have access to the GitHub Student Developer Pack[19] (for students) or GitHub Teacher Toolbox[20] (for faculty), which includes numerous free tools and services for learning and development.

11.1.2 GitHub Copilot Access for UC Davis Members

GitHub Copilot is an AI-powered coding assistant that can significantly accelerate your work. As a member of the UC Davis GitHub organization, you may be eligible for a free Copilot seat.

To request a Copilot seat:

If you are not yet a member of the UC Davis GitHub organization:

1. Ensure you have a GitHub account and have signed up for GitHub Education (see above)
2. Go to the UC Davis IT ServiceHub GitHub page[21]
3. Click the blue "Get GitHub!" button near the top right
4. When you receive the invitation email, make sure to check the "Ask for a GitHub Copilot seat (optional)" checkbox before joining (Figure 11.1)

[18] https://education.github.com/
[19] https://education.github.com/pack
[20] https://education.github.com/teachers
[21] https://servicehub.ucdavis.edu/servicehub?id=it_catalog_content&sys_id=951add951b5798103f4286ae6e4bcb12

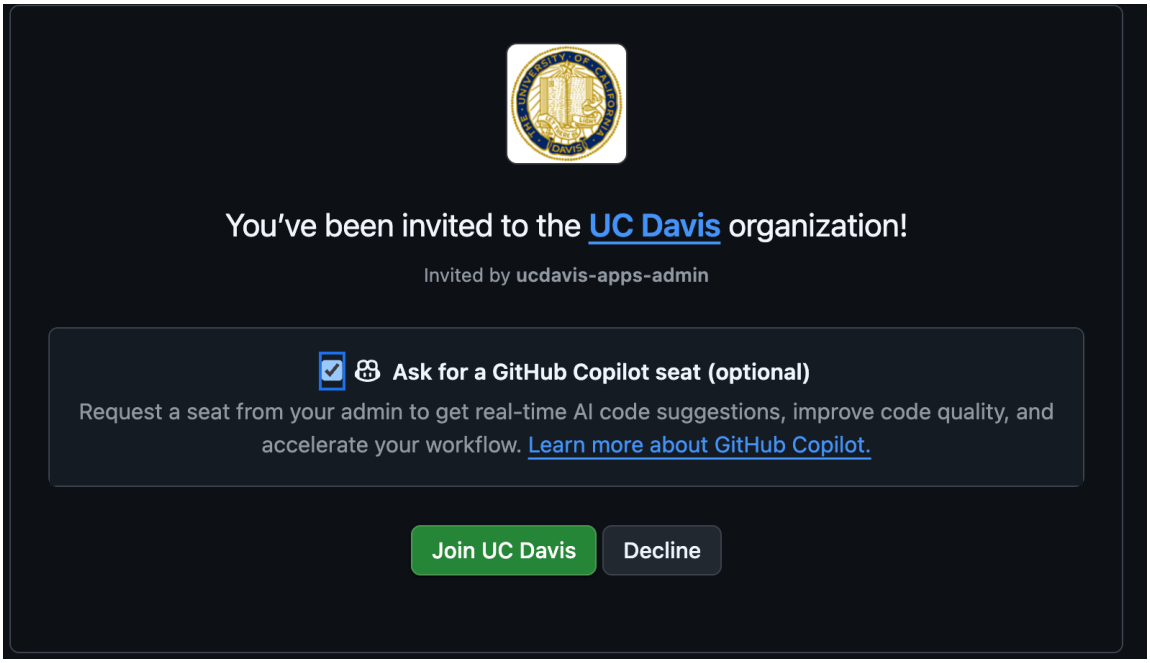


Figure 11.1: UC Davis GitHub invitation with Copilot seat option

5. Click "Join UC Davis" to accept the invitation
6. Follow the setup instructions to install and configure Copilot in your development environment

If you are already a member of the UC Davis GitHub organization:

1. Go to your GitHub Copilot settings[22]
2. Scroll to the "Get Copilot from an organization" section at the bottom
3. Find the "ucdavis" entry and click the request button

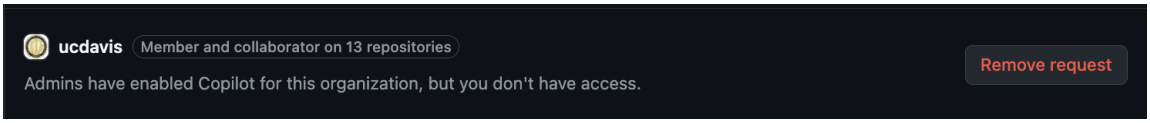


Figure 11.2: GitHub Copilot settings page showing organization options

4. Wait for approval from the UC Davis GitHub organization administrators
5. Once approved, follow the setup instructions to install and configure Copilot in your development environment

For guidance on using GitHub Copilot effectively, see the wai site's "Best Practices for Safe and Successful Use" section[23].

[22] https://github.com/settings/copilot/features
[23] https://morrison-lab.github.io/wai/chapters/coding-agents.html#sec-ai-best-practices

12 Collaborative Software Development

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Collaborative Software Development**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Pull Requests[3]
- Code Review[4]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/collaborative-software-development.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/collaborative-software-development.html
[3] https://morrison-lab.github.io/psc/chapters/collaborative-software-development.html#sec-pull-requests
[4] https://morrison-lab.github.io/psc/chapters/collaborative-software-development.html#sec-code-review

13 Continuous Integration

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Continuous Integration**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Understanding GitHub Actions[3]
- Setting Up GitHub Actions[4]
- Reusable Workflow Collections[5]
- How GitHub Actions Workflows Work[6]
- Workflow Files and Security[7]
- Pipeline Design Patterns[8]
- Troubleshooting Failed Workflows[9]
- Pull Request Comment Automation[10]
- Fast Package Installation with r2u[11]
- Additional Resources[12]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/continuous-integration.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/continuous-integration.html
[3] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-understanding-github-actions
[4] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-setting-up-github-actions
[5] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-reusable-workflow-collections
[6] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-how-github-actions-workflows-work
[7] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-workflow-files-and-security
[8] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-ci-design-patterns
[9] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-troubleshooting-failed-workflows
[10] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-pr-comment-automation
[11] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-fast-package-installation-with-r2u
[12] https://morrison-lab.github.io/psc/chapters/continuous-integration.html#sec-continuous-integration-additional-resources

14 Unix

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Unix**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- User interfaces: CLI, TUI, and GUI[3]
- Basics[4]
- Terminal Emulators[5]
- Syntax for both Mac/Windows[6]
- Zsh and Oh My Zsh[7]
- Running Bash Scripts[8]
- Shell Scripting Style[9]
- Running Rscripts in Windows[10]
- Checking tasks and killing jobs[11]
- Running big jobs[12]
- External Resources[13]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/unix.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/unix.html
[3] https://morrison-lab.github.io/psc/chapters/unix.html#sec-cli-tui-gui
[4] https://morrison-lab.github.io/psc/chapters/unix.html#sec-unix-basics
[5] https://morrison-lab.github.io/psc/chapters/unix.html#sec-terminal-emulators
[6] https://morrison-lab.github.io/psc/chapters/unix.html#sec-syntax-for-both-macwindows
[7] https://morrison-lab.github.io/psc/chapters/unix.html#sec-zsh
[8] https://morrison-lab.github.io/psc/chapters/unix.html#sec-running-bash-scripts
[9] https://morrison-lab.github.io/psc/chapters/unix.html#sec-shell-style
[10] https://morrison-lab.github.io/psc/chapters/unix.html#sec-running-rscripts-in-windows
[11] https://morrison-lab.github.io/psc/chapters/unix.html#sec-checking-tasks-and-killing-jobs
[12] https://morrison-lab.github.io/psc/chapters/unix.html#sec-running-big-jobs
[13] https://morrison-lab.github.io/psc/chapters/unix.html#sec-external-resources

15 Reproducible Environments

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Reproducible Environments**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Package Version Control with renv[3]
- Fast Package Installation on Ubuntu with r2u[4]
- Managing R Versions with rig[5]
- Clearing Docker Cache[6]

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html
[3] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html#sec-package-version-control-with-renv
[4] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html#sec-fast-package-installation-on-ubuntu-with-r2u
[5] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html#sec-managing-r-versions-with-rig
[6] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html#sec-clearing-docker-cache

16 Code Publication

This chapter has moved to *Principles of Scientific Computing*[1], the Morrison Lab's general-purpose guide to coding for research projects: **Code Publication**[2] (source: https://github.com/Morrison-Lab/psc).

That page covers:

- Checklist overview[3]
- Fill out file headers[4]
- Clean up comments[5]
- Document functions[6]
- Remove deprecated filepaths[7]
- Ensure project runs via bash[8]
- Complete the README[9]
- Clean up feature branches[10]
- Create GitHub release[11]
- Releasing to CRAN[12]

> **Looking for a specific section that used to be here?**
>
> This chapter's old section anchors still resolve on this page, so old bookmarks and links land here, but the content now lives on the page linked above, under the same anchor. For example, `#sec-foo` here is `chapters/code-publication.html#sec-foo` on the psc site.

[1] https://morrison-lab.github.io/psc/
[2] https://morrison-lab.github.io/psc/chapters/code-publication.html
[3] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-checklist-overview
[4] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-fill-out-file-headers
[5] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-clean-up-comments
[6] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-document-functions
[7] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-remove-deprecated-filepaths
[8] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-ensure-project-runs-via-bash
[9] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-complete-the-readme
[10] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-clean-up-feature-branches
[11] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-create-github-release
[12] https://morrison-lab.github.io/psc/chapters/code-publication.html#sec-cran-release

17 Data Publication

17.1 Overview

Warning! ***NEVER*** *push a dataset into the public domain (e.g., GitHub, OSF) without first checking with lab leadership to ensure that it is appropriately de-identified and we have approval from the sponsor and/or human subjects review board to do so. For example, we will need to re-code participant IDs (even if they contain no identifying information) before making data public to completely break the link between IDs and identifiable information stored on our servers.*

If you are releasing data into the public domain, then consider making available *at minimum* a `.csv` file and a codebook of the same name (note: you should have a codebook for internal data as well). We often also make available `.rds` files as well. For example, your `mystudy/data/public` directory could include three files for a single dataset, two with the actual data in `.rds` and `.csv` formats, and a third that describes their contents:

```
analysis_data_public.csv
analysis_data_public.rds
analysis_data_public_codebook.txt
```

In general, datasets are usually too big to save on GitHub, but occasionally they are small. Here is an example of where we actually pushed the data directly to GitHub: https://github.com/ben-arnold/enterics-seroepi/tree/master/data .

If the data are bigger, then maintaining them under version control in your git repository can be unwieldy. Instead, we recommend using another stable repository that has version control, such as the Open Science Framework ("Open Science Framework," n.d.). For example, all of the data from the WASH Benefits trials (led by investigators at Berkeley, icddr,b, IPA-Kenya and others) are all stored through data components nested within in OSF projects: https://osf.io/tprw2/. Another good option is Dryad Digital Repository ("Dryad Digital Repository," n.d.) or institutional digital repositories.

We recommend cross-linking public files in GitHub (scripts/notebooks only) and OSF/Dryad/institutional digital repositories.

Below are the main steps to making data public, after finalizing the analysis datasets and scripts:
1. Remove Protected Health Information (PHI)
2. Create public IDs or join already created public IDs to the data

3. Create an OSF repository and/or Dryad/institutional digital repository
4. Edit analysis scripts to run using the public datasets and test (optional)
5. Create a public github page for analysis scripts and link to OSF and/or Dryad/Zenodo
6. Go live

17.2 Removing PHI

Once the data is finalized for analysis, the first step is to strip it of Protected Health Information (PHI), or any other data that could be used to link back to specific participants, such as names, birth dates, or GPS coordinates at the village/neighborhood level or below. PHI includes, but is not limited to:

17.2.1 Personal information

These are identifiers that directly point to specific individuals, such as:
- Names, addresses, photographs, date of birth
- A combination of age, sex, and geographic location (below population 20,000) is considered identifiable

17.2.2 Dates

Any specific dates (e.g., study visit dates, birth dates, treatment dates) are usually problematic.
- If a dataset requires high resolution temporal information, coarsen visit or measurement dates to be two variables: year and week of the year (1-52).
- If a dataset requires age, provide that information without a birth date (typically month resolution is sufficient)

Caution! *If making changes to the format of dates or ages, make sure your analysis code runs on these modified versions of the data (step 3)!*

17.2.3 Geographic information

Do not include GPS coordinates (longitude, latitude) except in special circumstances where they have been obfuscated/shifted. Reach out to lab leadership before doing this because it can be complicated.

Do not include place names or codes (e.g., US Zip Codes) if the place contains <20,000 people. For villages or neighborhoods, code them with uninformative IDs. For sub-districts or districts, names are fine.

If an analysis requires GPS locations (e.g., to make a map), then typically we include a disclaimer in the article's data availability statement that explains we cannot make GPS locations public to protect participant confidentiality. As a middle ground, we typically

make our *code* public that runs on the geo-located data for transparency, even if independent researchers can't actually run that code (although please be careful to ensure the code itself does not in any way include geographic identifiers).

For more examples of what constitutes PHI, please refer to this link: https://cphs.berkeley.edu/hipaa/hipaa18.html

For learning about working with geospatial data, see the UC Davis DataLab GIS workshops[1], including resources on QGIS and spatial SQL.

17.3 Create public IDs

17.3.1 Rationale

The UC Davis IRB requires that public datasets not include the original study IDs to identify participants or other units in the study. The reason is that those IDs are linked in our private datasets to PHI. By creating a new set of public IDs, the public dataset is one step further removed from the potential to link to PHI.

17.3.2 A single set of public IDs for each study

For each study, it is ideal to create a single set of public IDs whenever possible. We could create a new set of public IDs for every public dataset, but the downside is that independent researchers could no longer link data that might be related. By creating a single set of public IDs associated with each internal study ID, public files retain the link.

Maintaining a single set of public IDs requires a shared "bridge" dataset, that includes a row for each study ID and has the associated public ID. For studies with multiple levels of ID, we would typically have separate bridge datasets for each type of ID (e.g,. cluster ID, participant ID, etc.)

Create a public ID that can be used to uniquely identify participants and that can internally be linked to the original study IDs. We recommend creating a subdirectory in the study's shared data directory to store the public IDs. The shared location enables multiple projects to use the same IDs. Create the IDs using a script that reads in the study IDs, creates a unique (uninformative) public ID for the study IDs, and then saves the bridge dataset. The script should be saved in the same directory as the public ID files.

Caution! *Note that small differences may arise if the new public IDs do not necessarily order participants in the same way as the internal IDs. The small differences are all in estimates that rely on resampling, such as Bootstrap CIs, permutation P-values, and TMLE, as the resampling process may lead tp slightly different re-samples. The key here, to ensure the results are consistent irrespective of the dataset used, is simply to not assign public IDs randomly. Use* `rank()` *on the internal ID instead of* `row_number()` *to ensure that the order is always the same.*

---

[1] https://github.com/ucdavisdatalab

17.3.3 Example scripts

Please see the reproducible example provided by the Proctor Lab at UCSF[2] that you can run and replicate when making data public for your projects. The example contains the following files and folders:

1. `data/final/`- folder containing the projects final data in both csv and rds formats

2. `code/DEMO_generate_public_IDs.R`- creates randomly generated public IDs that can be matched to the trial's assigned patient IDs.

3. `data/make_public/DEMO_internal_to_publicID.csv`- the output from step #2, a bridge dataset with two variables- the new public ID and the patient's assigned ID.

4. `code/DEMO_create_public_datasets.R`- joins the public IDs to the trial's full dataset, and strips it of the assigned patient ID.
5. `data/public/`- folder containing the output from step #3- de-identified public dataset, in csv and rds formats, with uniquely identifying public IDs that cannot be easily linked back to the patient's ID.

The example workflow is accessible via GitHub: https://github.com/proctor-ucsf/dcc-handbook/tree/master/templates/making-data-public

17.4 Create a data repository

First, ensure that you create a codebook and metadata file for each public dataset (see the DCC guide on Documenting datasets[3]). Use the same name as the datasets, but with `-codebook.txt`, `-codebook.html`, or `-codebook.csv` at the end (depending on the file format for the codebook). One option is the R `codebook` package, which also generates JSON output that is machine-readable.

For modern, standardized data dictionaries, consider the Tidyverse data-dict[4] project. The `data-dict.yaml` specification defines an open, lightweight, plain-text YAML format for describing tabular datasets—including table schemas, column data types, descriptions, relationships, and validation constraints. Because data dictionaries are defined in plain-text YAML files, they can be version-controlled directly in Git alongside your analysis code. The companion `data-dict` command-line tool can draft dictionaries from existing data files, validate dataset values against your dictionary rules, and render self-contained HTML documentation reports.

For additional guidance on data documentation best practices, see the UC Davis DataLab workshop on data documentation[5].

[2] https://proctor-ucsf.github.io/dcc-handbook/publicdata.html
[3] https://proctor-ucsf.github.io/dcc-handbook/datawrangling.html#documenting-datasets
[4] https://data-dict.tidyverse.org/
[5] https://github.com/ucdavisdatalab/workshop_how-to-data-documentation

17.4.1 Steps for creating an Open Science Framework (OSF) repository:

1. Create a new OSF project per these instructions: https://help.osf.io/article/252-create-a-project
2. Create a data component and upload the datasets in .csv and .rds format along with the codebooks. The primary format for public dissemination is .csv but we make the .rds files available too as auxiliary files for convenience.
3. Create a notebook component and upload the final .html files (which will not be on github… but see optional item below)
4. On the OSF landing Wiki, provide some context. Here is a recent example: https://osf.io/954bt/
5. Create a Digital Object Identifier (DOI) for the repository. A DOI is a unique identifier that provides a persistent link to content, such as a dataset in this case. Learn more about DOIs[6]
6. Optional: Complete the software checklist and system requirement guide for the analysis to guide others. Include it on the GitHub README for the project: https://github.com/proctor-ucsf/mordor-antibody

17.5 Edit and test analysis scripts

Make minor changes to the analysis scripts so that they run on public data. If using version control in GitHub, the most straight-forward way is to create a branch from the main git branch that reads in the public files, and then renames the new public ID variable, e.g., "id_public" to the internally recognized ID variable name, e.g. "recordID", when reading in the public data. Re-run all the analysis scripts to ensure that they still work with the public version of the dataset.

17.6 Create a public GitHub page for public scripts

At minimum, we should include all of the scripts required to run the analyses. **IMPORTANT**: ensure you have taken a snapshot and saved your computing environment using the `renv` package (see Reproducible Environments[7]).

See examples:
- ACTION - https://github.com/proctor-ucsf/ACTION-public
- NAITRE - https://github.com/proctor-ucsf/NAITRE-primary

> ***Caution!*** *Read through the scripts carefully to ensure there is no PHI in the code itself*

Once a public GitHub page exists, you can create a new component on an OSF project (step 3, above) and link it to the public version of the GitHub repo.

[6] https://researchdata.princeton.edu/research-lifecycle-guide/publishing-and-preservation/dois
[7] https://morrison-lab.github.io/psc/chapters/reproducible-environments.html

17.7 Go live

On GitHub, it is useful to create an official "release" version to freeze the repository, where you can have "associated files" with each version. Include the .html notebook output as additional files — since they aren't tracked in GitHub, it does provide a way of freezing / saving the HTML output for us and others. OSF examples of studies from UCSF's Proctor Foundation:

- ACTION - https://osf.io/ca3pe/
- NAITRE - https://osf.io/ujeyb/
- MORDOR Niger antibody study - https://osf.io/dgsq3/

Further reading on end-to-end data management: How to Store and Manage Your Data - PLOS ("How to Store and Manage Your Data," n.d.)

18 High-performance computing (HPC)

When you need to run a script that requires a large amount of RAM, large files, or that uses parallelization, UC Davis provides several high-performance computing (HPC) resources.

18.1 UC Davis Computing Resources

18.1.1 Available Resources

UC Davis HPC Clusters: - **Farm Cluster** (hpc.ucdavis.edu[1]): UC Davis's primary HPC cluster providing shared computing resources for research

PHS Shared Compute Environments: For lab members affiliated with the School of Public Health Sciences (PHS), additional shared computing environments are available. These environments provide secure, HIPAA-compliant computing resources suitable for working with sensitive health data.

- **Shiva** (shiva.ucdavis.edu): SLURM-based cluster for computational work
- **Mercury** (mercury.ucdavis.edu): RStudio GUI computing environment

For detailed information about PHS shared compute environments, including access procedures, security guidelines, and usage policies, please refer to the PHS Shared Compute Environments Guide[2].

Contact lab leadership for assistance with: - Requesting access to computing resources - Choosing the appropriate computing environment for your project - Setting up your computing environment

18.2 Getting started with SLURM clusters

To access a UC Davis HPC cluster, in terminal, log in using SSH. For example, to access shiva:

```
ssh USERNAME@shiva.ucdavis.edu
```

You will be prompted to enter your UC Davis credentials and may need to complete two-factor authentication.

Once you log in, you can view the contents of your home directory in command line by entering `cd $HOME`. You can create subfolders within this directory using the `mkdir` command. For example, you could make a "code" subdirectory and clone a Github repository there using the following code:

[1] https://hpc.ucdavis.edu/
[2] assets/files/PHS_Shared_Compute_Environments.pdf

```
cd $HOME
mkdir code
git clone https://github.com/jadebc/covid19-infections.git
```

18.2.1 One-Time System Set-Up

To keep the install packages consistent across different nodes, you will need to explicitly set the pathway to your R library directory.

Open your `~/.Renviron` file (`vi ~/.Renviron`) and append the following line:

Note: Once you open the file using `vi [file_name]`, you must press `i` (on Mac OS) or `Insert` (on Windows) to make edits. After you finish, hit `Esc` to exit editing mode and type `:wq` to save and close the file.

```
R_LIBS=~/R/x86_64-pc-linux-gnu-library/4.0.2
```

Alternatively, run an R script with the following code on the cluster:

```
r_environ_file_path = file.path(Sys.getenv("HOME"), ".Renviron")
if (!file.exists(r_environ_file_path)) file.create(r_environ_file_path)

cat("\nR_LIBS=~/R/x86_64-pc-linux-gnu-library/4.0.2",
    file = r_environ_file_path, sep = "\n", append = TRUE)
```

To load packages that run off of C++, you'll need to set the correct compiler options in your R environment.

Open the Makevars file (`vi ~/.R/Makevars`) and append the following lines

```
CXX14FLAGS=-O3 -march=native -mtune=native -fPIC
CXX14=g++
```

Alternatively, create an R script with the following code, and run it on the cluster:

```
dotR = file.path(Sys.getenv("HOME"), ".R")
if (!file.exists(dotR)) dir.create(dotR)

M = file.path(dotR, "Makevars")
if (!file.exists(M)) file.create(M)

cat("\nCXX14FLAGS=-O3 -march=native -mtune=native -fPIC",
    "CXX14=g++",
    file = M, sep = "\n", append = TRUE)
```

18.3 Moving files to the cluster

The `$HOME` directory is a good place to store code and small test files. Save large files to the `$SCRATCH` directory or other designated storage areas. Check with the UC Davis HPC documentation[3] for specific quotas and retention policies. It's best to create a bash script that records the file transfer process for a given project. See example code below:

```
# note: the following steps should be done from your local
# (not after ssh-ing into the cluster)

# securely transfer folders from Box to cluster home directory
# note: the -r option is for folders and is not needed for files
scp -r "Box/project-folder/folder-1/" USERNAME@shiva.ucdavis.edu:/home/users/USERNAME/

# securely transfer folders from Box to your cluster scratch directory
scp -r "Box/project-folder/folder-2/" USERNAME@shiva.ucdavis.edu:/scratch/users/USERNAME/

# securely transfer folders from Box to shared scratch directory
scp -r "Box/project-folder/folder-3/" USERNAME@shiva.ucdavis.edu:/scratch/group/GROUPNAME/
```

18.4 Installing packages on the cluster

When you begin working on a cluster, you will most likely encounter problems with installing packages. To install packages, login to the cluster on the command line and open a development node. Do not attempt to do this in RStudio Server, as you will have to re-do it for every new session you open.

```
ssh USERNAME@shiva.ucdavis.edu

sdev
```

You should only have to install packages once. The cluster may require that you specify the repository where the package is downloaded from. You may also need to add an additional argument to install.packages to prevent the packages from locking after installation:

```
install.packages(<PACKAGE NAME>, repos="https://cran.r-project.org",
                 INSTALL_opts = "--no-lock")
```

In order for some R packages to work on clusters, it is necessary to load specific software modules before running R. These must be loaded each time you want to use the package in R. For example, for spatial and random effects analyses, you may need the modules/packages below. These modules must also be loaded on the command line prior to opening R in order for package installation to work.

[3] https://hpc.ucdavis.edu/

```
module --force purge # remove any previously loaded modules, including math and devel
module load math
module load math gmp/6.1.2
module load devel
module load gcc/10
module load system
module load json-glib/1.4.4
module load curl/7.81.0
module load physics
module load physics udunits geos
module load physics gdal/2.2.1 # for R/4.0.2
module load physics proj/4.9.3 # for R/4.0.2
module load pandoc/2.7.3

module load R/4.0.2

R # Open R in the Shell window to install individual packages or test code
Rscript install-packages.R # Alternatively, run a package installation script in the Shell
```

Figuring out the issues with some packages will require some trial and error. If you are still encountering problems installing a package, you may have to install other dependencies manually by reading through the error messages. If you try to install a dependency from CRAN and it isn't working, it may be a module. You can search for it using the `module spider` command:

```
module spider DEPENDENCY NAME
```

You can also reach out to UC Davis HPC support for help. Visit hpc.ucdavis.edu[4] for support information.

18.5 Testing your code

Both of the following ways to test code on a cluster are recommended for making small changes, such as editing file paths and making sure the packages and source files load. You should write and test the functionality of your script locally, only testing on the cluster once major bugs are out.

18.5.1 The command line

There are two main ways to explore and test code on computing clusters. The first way is best for users who are comfortable working on the command line and editing code in base R. Even if you are not comfortable yet, this is probably the better way because these commands will transfer between different cluster computers using Slurm.

Typically, you will want to initially test your scripts by initiating a development node using the command `sdev`. This will allocate a small amount of computing resources for 1 hour. You can access R via command line using the following code.

[4] https://hpc.ucdavis.edu/

```
# open development node
sdev

# Load all the modules required by the packages you are using
module load MODULE NAME

# Load R (default version)*
module load R

# initiate R in command line
R
```

*Note: for collaboration purposes, it's best for everyone to work with one version of R. Check what version is being used for the project you are working on. Some packages only work with some versions of R, so it's best to keep it consistent.

18.5.2 RStudio Server

For RStudio GUI computing, UC Davis provides mercury.ucdavis.edu. This is accessed through a web browser and provides an RStudio interface. You will be prompted to authenticate with your UC Davis credentials. This is the best way to work with R for people who are not comfortable accessing & editing in base R in a Shell application.

Note that mercury does not have SLURM, so it's best suited for interactive work and smaller computations. For large-scale computations requiring SLURM job scheduling, use shiva.ucdavis.edu instead.

When using RStudio Server, you can test your code interactively. However, do NOT use the RStudio Server's Terminal to install packages and configure your environment for SLURM-based clusters, as you will likely need to re-do it for every session/project. For SLURM clusters, use the command line approach described earlier.

18.5.3 Filepaths & configuration on the cluster

In most cases, you will want to test that the file paths work correctly on the cluster. You will likely need to add code to the configuration file in the project repository that specifies cluster-specific file paths. Here is an example:

```
# set cluster-specific file paths
if(Sys.getenv("LMOD_SYSHOST")!=""){

  cluster_path = paste0(Sys.getenv("HOME"), "/project-name/")

  data_path = paste0(cluster_path, "data/")
  results_path = paste0(cluster_path, "results/")
}
```

18.6 Running graphical programs with X11 forwarding

Shiva's SSH server permits X11 forwarding (checked August 2026), so a graphical program started on the cluster can draw its window on your own computer. This is worth setting up when you want a GUI text editor on the cluster, or a web browser that sees the network from Shiva's position rather than from yours.

X11 forwarding sends every window update across the network, so it suits a text editor much better than it suits a browser. When all you want is to view a web page that Shiva itself is serving, such as an RStudio Server[5] session or a rendered report, forward the port instead and use the browser already on your own computer:

```
# from your own computer, not from Shiva
ssh -L 8787:localhost:8787 USERNAME@shiva.ucdavis.edu
```

With that running, `http://localhost:8787` in your local browser reaches the service on Shiva. This is far faster than forwarding a browser window, because only the page data crosses the network rather than every repainted pixel.

18.6.1 Install an X server on your own computer

The cluster draws into an X server, and that server runs on your machine rather than on Shiva. macOS and Windows do not ship one, so install the software listed in Table 18.1 before your first attempt.

Table 18.1: Where to get an X server, by operating system

Your computer	What to install
macOS	XQuartz[6]. Log out and back in after installing, so that `DISPLAY` is set.
Windows	VcXsrv[7], or MobaXterm[8], which bundles an X server together with an SSH client.
Windows, using WSL2	Nothing. WSLg[9] supplies the X server.
Linux	Usually nothing. An X11 session is itself an X server; a Wayland session relies on Xwayland[10] for X11 compatibility. Confirm `DISPLAY` is set locally before you connect.

18.6.2 Connect with X11 forwarding enabled

Ask for trusted X11 forwarding with `-Y`:

[5] https://posit.co/products/open-source/rstudio-server/
[6] https://www.xquartz.org/
[7] https://sourceforge.net/projects/vcxsrv/
[8] https://mobaxterm.mobatek.net/
[9] https://learn.microsoft.com/en-us/windows/wsl/tutorials/gui-apps
[10] https://wiki.archlinux.org/title/Wayland

```
ssh -Y USERNAME@shiva.ucdavis.edu
```

Use `-Y` rather than `-X` here. `-X` requests *untrusted* forwarding, and by default OpenSSH refuses untrusted X11 connections made more than twenty minutes into the session ("Ssh_config(5) Manual Page," n.d.). Any window already on screen keeps working, so the symptom is that a program you launch later in the session simply never appears. Untrusted forwarding also restricts clients under the X11 SECURITY extension, to stop them reading or tampering with data belonging to trusted clients ("Ssh_config(5) Manual Page," n.d.), and some graphical toolkits fail outright under those restrictions.

The exception is a Linux client running Debian or Ubuntu, where `ForwardX11Trusted` defaults to `yes` and `-X` therefore already means the same thing as `-Y` ("Ssh_config(5) Manual Page, Debian Edition," n.d.). Passing `-Y` is correct on every platform, so prefer it and do not worry about which case you are in.

Trusted forwarding is a real tradeoff rather than a free upgrade. It gives programs running on Shiva full access to your local display, which includes reading input directed at your other windows ("Ssh_config(5) Manual Page," n.d.). That is an acceptable risk for a cluster our lab administers and trusts; do not extend the same setting to machines you do not trust.

To avoid retyping the flag, add an entry to `~/.ssh/config` on your own computer:

```
Host shiva
  HostName shiva.ucdavis.edu
  User USERNAME
  ForwardX11 yes
  ForwardX11Trusted yes
  Compression yes
```

`ssh shiva` then forwards X11 automatically. `Compression yes` is worth setting because X11 traffic compresses well over a slow link.

18.6.3 Check that the forward is working

After logging in, `DISPLAY` should be set:

```
echo $DISPLAY
# localhost:10.0
```

An empty result means the forward did not happen. The usual causes are omitting `-Y`, or not having an X server running on your own computer.

To confirm that the connection reaches your display, ask the display to describe itself:

```
xdpyinfo | head -3
```

Use `xdpyinfo` rather than `xeyes` for this test. Shiva has the `x11-utils` package, which provides `xdpyinfo`, but not `x11-apps`, which provides `xeyes` and `xclock` (checked August 2026).

18.6.4 Install graphical programs without `sudo`

Lab accounts on Shiva are not in the `sudo` group, which held only a handful of administrator accounts when this was checked in August 2026. Run `groups` to see which groups your own account belongs to. This means `apt install` is unavailable, and most installation instructions written for Ubuntu will not apply. Install into your home directory instead, which needs no special privileges.

Note that this is necessary even for programs that appear to be present already. The system Emacs at `/usr/bin/emacs` is the `emacs-nox` build, which links no X libraries at all and so cannot open a window under any circumstances. No web browser is installed on Shiva. `module avail` on the login node lists only MPI and numerical libraries, so it supplies neither one.

For Emacs, use conda-forge[11], whose build includes GTK and X11 support. If you do not already have conda, install Miniforge[12] into your home directory first.

```
conda create -y -n gui -c conda-forge emacs
conda activate gui
emacs &
```

For Firefox, use Mozilla's official Linux build, which is a self-contained directory that runs from anywhere:

```
mkdir -p ~/opt && cd ~/opt
curl -L -o firefox.tar.xz \
  'https://download.mozilla.org/?product=firefox-latest-ssl&os=linux64&lang=en-US'
tar xf firefox.tar.xz && rm firefox.tar.xz
~/opt/firefox/firefox &
```

Adding `~/bin` to your `PATH` and putting small wrapper scripts there saves typing the full path each time.

18.6.5 Limitations

R cannot open plot windows on Shiva. The installed R is built without X11 support, which you can confirm with `capabilities()["X11"]`. No amount of forwarding configuration changes this, because the R binary itself has no X11 device compiled in. Write plots to a file with `{ragg}`[13] or `ggplot2::ggsave()` and copy them back with `scp`, or forward a port and use a browser-based graphics device such as `{httpgd}`[14].

The faster remote-desktop tools are unavailable. Xpra[15], VNC, and X2Go[16] all compress far better than plain X11 forwarding over a slow link. All of them need an X server installed on the cluster itself, which requires root, so none can be set up from a lab account. Contact the cluster administrators if the performance of plain forwarding is blocking your work.

[11] https://conda-forge.org/
[12] https://github.com/conda-forge/miniforge
[13] https://ragg.r-lib.org/
[14] https://cran.r-project.org/package=httpgd
[15] https://github.com/Xpra-org/xpra
[16] https://wiki.x2go.org/

18.7 Recovering a disconnected interactive session

Interactive sessions on a SLURM cluster — a development node opened with `sdev`, or a session started with `srun --pty` ("Slurm Workload Manager," n.d.-e) or `salloc` ("Slurm Workload Manager," n.d.-a) — are tied to the terminal that launched them. If your SSH connection drops (a broken pipe, a network blip, or closing your laptop), SLURM usually sends the launching process a hang-up signal and **tears down the whole job allocation**, not just your shell. When that happens there is nothing left to reconnect to, and any unsaved work in that session is lost.

There is no way to reattach to the *identical* shell once the allocation has been released. The reliable strategy is to prevent the drop in the first place; if the allocation does happen to survive, you can open a *new* shell inside it.

18.7.1 Prevent drops with a terminal multiplexer (recommended)

Start a terminal multiplexer such as `tmux`[17] or `screen`[18] **on the login node**, then launch your interactive session from inside it:

```
ssh USERNAME@shiva.ucdavis.edu

# start a tmux session on the login node
tmux

# request your interactive allocation from inside tmux
sdev          # or: srun --pty bash
```

Because `tmux` keeps running on the login node even after your SSH connection drops, the launching process survives and the allocation stays alive. When you reconnect, reattach to your multiplexer and you are back where you left off:

```
ssh USERNAME@shiva.ucdavis.edu
tmux ls         # list existing sessions
tmux attach     # reattach to the most recent session
```

18.7.2 Reconnect to a surviving allocation

If you did not use a multiplexer, you can still open a new shell on the nodes of a job that is **already running** — most commonly a batch job submitted with `sbatch`, or an interactive allocation that happened to survive. This gives you a *new* shell (a new job step), not the original session.

First, find the job ID ("Slurm Workload Manager," n.d.-d):

```
squeue --me        # or: squeue -u $USER
```

[17] https://github.com/tmux/tmux/wiki
[18] https://www.gnu.org/software/screen/

Then launch a new interactive step inside that allocation, replacing `JOBID` with the ID reported by `squeue` ("Slurm Workload Manager," n.d.-e):

```
srun --jobid=JOBID --overlap --pty bash
```

> **!** The `--overlap` flag is required on recent SLURM
>
> Since SLURM 20.11, a new job step requests exclusive use of the job's resources by default, so without `--overlap` the `srun` above will simply hang waiting for resources the running step already holds. Always include `--overlap` when attaching to a running job.

If you only need to reattach to the input and output streams of an existing step (rather than start a fresh shell), `sattach JOBID.STEPID` does that instead ("Slurm Workload Manager," n.d.-b).

> **i** Note
>
> Exact behavior varies by cluster: some sites disable `srun` into running jobs, wrap it in a local helper, or require different flags. When in doubt, check the UC Davis HPC documentation[a] or contact HPC support.
>
> [a] https://hpc.ucdavis.edu/

18.8 Storage & group storage access

18.8.1 Individual storage

There are multiple places to store your files on computing clusters. Each user has their own `$HOME` directory as well as a `$SCRATCH` directory. These are directories that can be accessed via the command line once you've logged in to the cluster:

```
cd $HOME
cd /home/users/USERNAME # Alternatively, use the full path

cd $SCRATCH
cd /scratch/users/USERNAME # Full path
```

You can also navigate to these using the File Explorer if available through a web interface.

`$HOME` typically has a volume quota (e.g., 15 GB). `$SCRATCH` typically has a larger volume quota (e.g., 100 TB), but files here may get deleted after a certain period of inactivity. Thus, use `$SCRATCH` for test files, exploratory analyses, and temporary storage. Use `$HOME` for long-term storage of important files and more finalized analyses.

Check with the UC Davis HPC documentation[19] for specific storage options and quotas.

[19] https://hpc.ucdavis.edu/

18.8.2 Group storage

The lab may have shared `$GROUP_HOME` and `$GROUP_SCRATCH` directories to store files for collaborative use. These typically have larger quotas and may have different retention policies. You can access these via the command line or navigate to them using the File Explorer:

```
cd $GROUP_HOME
cd /home/groups/GROUPNAME

cd $GROUP_SCRATCH
cd /scratch/groups/GROUPNAME
```

However, saving files to group storage can be tricky. You can try using the scp command in the section "Moving files to the cluster" to see if you have permission to add files to group directories. Read the next section to ensure any directories you create have the right permissions.

18.8.3 Folder permissions

Generally, when we put folders in `$GROUP_HOME` or `$GROUP_SCRATCH`, it is so that we can collaborate on an analysis within the research group, so multiple people need to be able to access the folders. If you create a new folder in `$GROUP_HOME` or `$GROUP_SCRATCH`, please check the folder's permissions to ensure that other group members are able to access its contents. For example, you can inspect a folder's permissions by running `ls -l` from the level above it. For a quick refresher on interpreting the permission bits and updating them with `chmod`, see this guide[20].

18.9 Running big jobs

Once your test scripts run successfully, you can submit an sbatch script for larger jobs. These are text files with a `.sh` suffix. Use a text editor like Sublime to create such a script. Documentation on sbatch options is available from Slurm Workload Manager ("Slurm Workload Manager," n.d.-c). Here is an example of an sbatch script with the following options:

- `job-name=run_inc`: Job name that will show up in the SLURM system
- `begin=now`: Requests to start the job as soon as the requested resources are available
- `dependency=singleton`: Jobs can begin after all previously launched jobs with the same name and user have ended.
- `mail-type=ALL`: Receive all types of email notification (e.g., when job starts, fails, ends)
- `cpus-per-task=16`: Request 16 processors per task. The default is one processor per task.
- `mem=64G`: Request 64 GB memory per node.
- `output=00-run_inc_log.out`: Create a log file called `00-run_inc_log.out` that contains information about the Slurm session

[20] https://www.chriswrites.com/how-to-change-file-permissions-using-the-terminal/

- `time=47:59:00`: Set maximum run time to 47 hours and 59 minutes. If you don't include this option, the cluster will automatically exit scripts after 2 hours of run time (default may vary by cluster).

The file `analysis.out` will contain the log file for the R script `analysis.R`.

```
#!/bin/bash

#SBATCH --job-name=run_inc
#SBATCH --begin=now
#SBATCH --dependency=singleton
#SBATCH --mail-type=ALL
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --mem=64G
#SBATCH --output=00-run_inc_log.out
#SBATCH --time=47:59:00

cd $HOME/project-code-repo/2-analysis/

module purge

# load R version 4.0.2 (required for certain packages)
module load R/4.0.2

# load gcc, a C++ compiler (required for certain packages)
module load gcc/10

# load software required for spatial analyses in R
module load physics gdal
module load physics proj

R CMD BATCH --no-save analysis.R analysis.out
```

To submit this job, save the code in the chunk above in a script called `myjob.sh` and then enter the following command into terminal:

```
sbatch myjob.sh
```

To check on the status of your job, enter the following code into terminal:

```
squeue -u $USERNAME
```

19 Working with AI

Our notes on working responsibly and effectively with AI coding assistants have moved to a dedicated site: **Working with AI**[1] (source: https://github.com/Morrison-Lab/wai).

That site covers:

- **Policies for Using AI**[2]: responsibility for validation, disclosure, attribution, and using AI for journal articles
- **Coding Agents**[3]: what language models, coding agents, and harnesses are; how to work with them; benefits, hazards, and best practices; and configuring your environment
- **Pull-Request Workflow with Agents**[4]: filing issues, claiming work, and driving a pull request to a clean, mergeable state

Lab members who use AI tools must still adhere to the guidelines described there.

> **Looking for a specific section that used to be here?**
>
> This chapter previously had section anchors like `#sec-ai-best-practices` or `#sec-ai-disclosure` for each subtopic. Those anchors still resolve on this page (so old bookmarks and links won't 404), but the content itself now lives on the linked wai pages above — follow the matching bullet to find it.

19.1 Institutional AI account access

19.1.1 UC Davis Health OpenAI access

For lab members affiliated with UC Davis Health (UCDH) or conducting health sciences research, enterprise OpenAI accounts are provisioned through the UCDH IT Self-Service (ITSS) ServiceNow catalog.

To request an account:

1. Navigate to the UCDH ServiceNow OpenAI catalog item[5].
2. Log in with your UC Davis Health credentials and complete Duo two-factor authentication if prompted.
3. Fill out the request form with your department, contact details, and business or research justification.

[1] https://morrison-lab.github.io/wai/
[2] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html
[3] https://morrison-lab.github.io/wai/chapters/coding-agents.html
[4] https://morrison-lab.github.io/wai/chapters/pr-workflow-with-agents.html
[5] https://ucdh.service-now.com/itss?id=sc_cat_item&sys_id=738dd2ac93eba650c4dbb1847aba100d

4. Submit the request for processing by UCDH IT.
5. Once approved, follow the instructions in the confirmation email to join the UC Davis Health OpenAI workspace.

19.1.2 UC Davis campus OpenAI access

For UC Davis campus affiliates, access to ChatGPT Edu is managed through Information and Educational Technology (IET) Aggie AI. See the IET Aggie AI ChatGPT Edu documentation[6] for setup instructions and eligibility details.

19.1.3 Data security and compliance

When using institutional AI accounts:

- Never submit Protected Health Information (PHI), personally identifiable information (PII), or restricted research data to AI tools unless authorized under an approved protocol.
- Follow all institutional data handling policies and the lab's guidelines on Working with AI[7].

[6] https://iet.ucdavis.edu/aggie-ai/ai-tools/chatgpt-edu
[7] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html

20 Checklists

20.1 Onboarding checklist

Welcome to the lab! New members should complete the following checklist during their first week:

- ☐ **Account setup**: Request and confirm access to GitHub (join the `UCD-SERG` organization), Microsoft Teams, UC Davis email, and lab Box and SharePoint folders.
- ☐ **Required trainings**: Complete required CITI Human Subjects Biomedical Group 1 training and share your certificate with lab leadership (see Section 2.3).
- ☐ **Read the lab manual**: Review lab culture and conduct (Chapter 2), then the coding style[1] and collaborative development[2] chapters of *Principles of Scientific Computing*.
- ☐ **Environment setup**: Configure your local R development environment, clone a starter repository, and run `renv::restore()` to verify reproducible package installation (see Chapter 15).
- ☐ **HPC access**: Confirm High-Performance Computing cluster access (Farm or Shiva) and verify SSH keys if your project involves cluster computing (see Chapter 18).
- ☐ **Team introductions**: Introduce yourself on Microsoft Teams and at the next weekly lab meeting.
- ☐ **Initial 1:1 meeting**: Schedule a one-on-one meeting with lab leadership to align on project objectives, onboarding timeline, and initial milestones.

20.2 Pre-analysis plan checklist

- ☐ Brief background on the study (a condensed version of the introduction section of the paper)
- ☐ Hypotheses / objectives
- ☐ Study design
- ☐ Description of data
- ☐ Definition of outcomes
- ☐ Definition of interventions / exposures
- ☐ Definition of covariates
- ☐ Statistical power calculation
- ☐ Statistical model description
- ☐ Covariate selection / screening
- ☐ Standard error estimation method
- ☐ Missing data analysis
- ☐ Assessment of effect modification / subgroup analyses
- ☐ Sensitivity analyses
- ☐ Negative control analyses

[1] https://morrison-lab.github.io/psc/chapters/coding-style.html
[2] https://morrison-lab.github.io/psc/chapters/collaborative-software-development.html

20.3 Code checklist

- ☐ Does the script run without errors?
- ☐ Is code self-contained within repo and/or associated Box folder?
- ☐ Is all commented out code / remarks removed?
- ☐ Does the header accurately describe the process completed in the script?
- ☐ Is the script pushed to its github repository?
- ☐ Does the code adhere to the coding style guide[3]?
- ☐ Are all warnings ignorable? Should any warnings be intentionally suppressed or addressed?

20.4 Manuscript checklist

This is adapted in part from How small changes to a paper can help to smooth the review process (White 2019).

- ☐ Have you completed the relevant reporting checklist, if applicable? (See EQUATOR Network ("EQUATOR Network," n.d.) for a collection of checklists)
- ☐ Are the study results within the manuscript replicable (i.e., if you rerun the code in the study's repository, the tables and figures will be exactly replicated?)
- ☐ Is a target journal selected?
- ☐ Is the title declarative, in other words, does it state the object/findings rather than suggest them?
- ☐ Is the word count of the manuscript close to the target journal's allowance?
- ☐ Does the manuscript adhere to the formatting guide of the target journal?
- ☐ Does the manuscript use a consistent voice (passive or active – usually active is preferred … pun intended)?
- ☐ Is each figure and table (including supplementary material) referenced in the main text?
- ☐ Is there a caption for each figure and table (including supplementary material)?
- ☐ Are tables/figures and supplementary material numbered in accordance with their appearance in the main text?
- ☐ Does the text use past tense if it is reporting research findings or future tense if it is a study protocol?
- ☐ Does the text avoid subjective wording (e.g., "interesting", "dramatic")?
- ☐ Does the text use minimal abbreviations, and are all abbreviations defined at first use?
- ☐ Does the text avoid directionless words? (e.g., instead of writing, 'Precipitation influences disease risk', write, 'Precipitation was associated with increased disease risk').
- ☐ Does the text avoid making causal claims that are not supported by the study design? Be careful about the words "effect", "increase", and "decrease", which are often interpreted as causal.
- ☐ Does the text avoid describing results with the word "significant", which can easily be confused with statistical significance? (see references on this topic here[4])
- ☐ Have you drafted author contributions? Do they follow the CRediT: Contributor Roles Taxonomy ("CRediT," n.d.) for author contributions?

[3] https://morrison-lab.github.io/psc/chapters/coding-style.html
[4] https://journals.lww.com/epidem/Fulltext/2001/05000/The_Value_of_P.2.aspx

20.5 Figure checklist

- ☐ Are the x-axis and y-axis labeled?
- ☐ If the figure includes panels, is each panel labeled?
- ☐ Are there sufficient numerical / text labels and breaks on the x-axis and y-axis?
- ☐ Is the font size appropriate (i.e., large enough to read, not so large that it distracts from the data presented in the figure?)
- ☐ Are the colors used colorblind friendly? See a colorblind-friendly palette here[5], a neat palette generator with colorblind options here[6], and an article on why this matters: The misuse of colour in science communication (Crameri et al. 2020)
- ☐ Are colors/shapes/line types defined in a legend?
- ☐ Are the legends and other labels easy to understand with minimal abbreviations?
- ☐ If there is overplotting, is transparency used to show overlapping data?
- ☐ Are 95% confidence intervals or other measures of precision shown, if applicable?

20.6 Offboarding checklist

When preparing to depart the lab, members should complete this offboarding checklist to ensure project continuity:

- ☐ **Code handoff**: Ensure all analysis code is committed, pushed to GitHub (`UCD-SERG` organization), and documented with a self-contained `README` and reproducible execution instructions (see Chapter 5).
- ☐ **Data archiving**: Archive or transfer large datasets, raw files, and simulation outputs per lab storage protocols (see Chapter 17 and Chapter 18). Ensure all shared Drive and Box datasets have clear data dictionaries and codebooks.
- ☐ **Repository ownership**: Transfer any remaining repository or branch administrative permissions to the `UCD-SERG` organization and lab leadership.
- ☐ **Publication plan**: Coordinate with lab leadership on outstanding manuscripts, co-authorship agreements, remaining revisions, and target submission timelines.
- ☐ **Account transition**: Coordinate with lab leadership to transition or revoke personal access to shared lab accounts, server credentials, and IRB protocols.
- ☐ **Equipment return**: Return any lab-issued equipment (keys, hardware, storage media) and report any known hardware or configuration issues.
- ☐ **Exit interview**: Complete a departure check-in with lab leadership to provide feedback and share forward contact details for future manuscript correspondence.

[5] http://www.cookbook-r.com/Graphs/Colors_(ggplot2)/#a-colorblind-friendly-palette
[6] https://medialab.github.io/iwanthue/?utm_source=Nature+Briefing&utm_campaign=2c68711076-briefing-dy-20211006&utm_medium=email&utm_term=0_c9dfd39373-2c68711076-44335685

21 Resources

General-purpose resources have moved to the Morrison Lab's sister sites:

- R, Git and GitHub, the command line, Python, Julia, and open science: the Resources page[1] of *Principles of Scientific Computing*[2]
- AI and coding agents: *Working with AI*[3]
- Writing: Further reading[4] in *Principles of Scientific Writing*[5]
- Presentations, posters, and scientific figures: Slides, posters, and figures[6] in *Principles of Scientific Writing*[7]

21.1 Resources for R

- DataCamp[8] - email Kristen for access

21.2 Professional advice

- Professional advice, especially for your first job[9]
- Team Public Health Substack[10]

21.3 Funding

- Building Your Funding Train[11]
- NIH Grant Writing Resources[12]

[1] https://morrison-lab.github.io/psc/chapters/resources.html
[2] https://morrison-lab.github.io/psc/
[3] https://morrison-lab.github.io/wai/
[4] https://morrison-lab.github.io/psw/chapters/intro.html#sec-further-reading
[5] https://morrison-lab.github.io/psw/
[6] https://morrison-lab.github.io/psw/chapters/slides-posters-figures.html
[7] https://morrison-lab.github.io/psw/
[8] https://www.datacamp.com/
[9] https://docs.google.com/document/d/1ckgRCcr7FFPyymyMA6Y_-cB_vftOyrrxT28c6gRg0PI/edit
[10] https://teampublichealth.substack.com/
[11] https://grantwriting.stanford.edu/resources/funding-train/
[12] https://www.niaid.nih.gov/grants-contracts/write-grant-application

21.4 Ethics and global health research

- Global Code of Conduct For Research in Resource-Poor Settings[13]
- Who is a global health expert? (Ojiako et al. 2023)
- Transforming Global Health Partnerships[14]

21.5 Comparable Lab Manuals

Exploring lab manuals from peer research groups helps us identify best practices, adapt proven onboarding workflows, and align our open science standards with the broader computational biology and epidemiology communities:

- Benjamin-Chung Lab Manual[15] (GitHub repository[16]) — The computational epidemiology manual from which our manual originally adapted its structure, covering reproducible R workflows, collaborative development, and computational research practices.
- Fay Lab Manual[17] (GitHub repository[18]) — An Openscapes-origin Quarto manual for quantitative fisheries ecology (UMass Dartmouth), featuring detailed offboarding checklists, departure handoff protocols, and an open science communication philosophy.
- ContextLab Lab Manual[19] (GitHub repository[20]) — The Contextual Dynamics Lab manual at Dartmouth, notable for requiring incoming lab members to make an interactive pull request to learn contribution workflows hands-on.
- Proctor-UCSF DCC Handbook[21] (GitHub repository[22]) — A comprehensive handbook for the Data Coordinating Center at UCSF, focusing on clinical trial data management, REDCap workflows, quality assurance, and public data curation in global health settings.
- Quantitative Marine Ecology Lab Manual[23] (GitHub repository[24]) — A Quarto-based manual for Easton White's research group at UNH, featuring role-specific administrative checklists and curated reading lists organized by career stage.

21.6 Reading and Watching List

- *Good Luck, Have Fun, Don't Die*[25] — A science fiction film directed by Gore Verbinski following a traveler from the future who recruits patrons at a Los Angeles diner to prevent a rogue artificial intelligence from destroying the world.

[13] https://www.globalcodeofconduct.org/
[14] https://link.springer.com/book/9783031537929
[15] https://jadebc.github.io/lab-manual/
[16] https://github.com/jadebc/lab-manual
[17] https://thefaylab.github.io/lab-manual/
[18] https://github.com/thefaylab/lab-manual
[19] https://contextlab.github.io/lab-manual/
[20] https://github.com/ContextLab/lab-manual
[21] https://proctor-ucsf.github.io/dcc-handbook/
[22] https://github.com/proctor-ucsf/dcc-handbook
[23] https://quantmarineecolab.github.io/lab-manual/
[24] https://github.com/quantmarineecolab/lab-manual
[25] https://en.wikipedia.org/wiki/Good_Luck,_Have_Fun,_Don%27t_Die

22 Professional Development

22.1 Mentoring Philosophy

We believe in individualized mentoring that supports each person's unique career goals. Effective mentoring requires:

- Regular, open communication between mentees and mentors
- Mutual respect and trust
- Clear expectations and goals
- Constructive feedback
- Support for both research and career development

22.2 Individual Development Plans

All graduate students and postdocs should maintain an Individual Development Plan (IDP) that outlines:

- Short-term and long-term career goals
- Skills to develop
- Training needs and opportunities
- Timeline and milestones
- Progress toward goals

Update your IDP at least annually and discuss it with your PI. Useful resources:

- myIDP - Individual Development Plan tool[1]
- NIH OITE Career Resources[2]

22.3 Navigating the Hidden Curriculum

The "hidden curriculum" refers to the unspoken norms, unwritten expectations, and cultural practices in graduate school and academic research that are rarely taught in formal coursework but are critical for long-term success and well-being. Demystifying these implicit rules ensures equitable access to academic opportunities for scholars from all backgrounds.

[1] https://myidp.sciencecareers.org/
[2] https://www.training.nih.gov/career-services/

22.3.1 Managing Up and Mentoring Relationships

Mentorship is a collaborative partnership. Taking active ownership of your mentoring relationships helps ensure you receive the guidance and feedback you need:

- **Set clear meeting agendas**: Send a brief bulleted agenda and updated materials at least 24 hours before 1-on-1 meetings.
- **Clarify priorities and timelines**: Agree on concrete milestones, deadlines, and expectations for feedback turnarounds.
- **Build a mentoring team**: Seek guidance from multiple mentors, including thesis committee members, peer mentors, senior graduate students, and external collaborators. No single mentor can meet all training, professional, and personal needs.

22.3.2 Normalizing Setbacks and Imposter Feelings

Graduate training involves tackling open-ended scientific problems where methods fail, experiments yield inconclusive results, and manuscripts face rejection:

- **Imposter feelings are common**: Experiencing self-doubt or feeling underqualified is a widely shared experience in high-rigor research environments, especially during early transitions. It reflects the challenging nature of scientific discovery, not personal inadequacy.
- **Analytical iterations are normal**: Bugs, refactored data pipelines, and revisions are standard components of the scientific process.
- **Establish healthy boundaries**: Sustainable productivity requires rest, structured time off, and hobbies outside academia. Seek out campus health and counseling resources when facing persistent stress.

22.3.3 Demystifying Publishing and Peer Review

Navigating scientific publication requires understanding the review lifecycle:

- **Rejection is a normal part of publishing**: In academic publishing, manuscripts frequently undergo multiple rounds of submission; rejection provides an opportunity to refine arguments and submit to a better-matched venue.
- **Approaching reviewer feedback**: Read peer reviews, step back for a day to process feedback, and draft systematic, point-by-point response letters using the guidance in Section 24.2.
- **Authorship conversations**: Initiate discussions regarding authorship order and contribution roles early in every project and revisit them regularly.

22.3.4 Strategic Networking and Visibility

Building professional connections expands research opportunities and career horizons:

- **Conferences as networking venues**: Prepare a clear 60-second summary of your research question, key methods, and scientific impact.
- **Engage with speakers**: Introduce yourself after departmental seminars or during poster sessions to ask focused questions about their work.

- **Peer networks**: Cultivate strong relationships with cohort members and lab peers; colleagues at your career stage will become future collaborators, reviewers, and lifelong allies.

22.4 Presentations and Conferences

We encourage lab members to present their work at conferences and seminars:

- Discuss conference opportunities with PIs early
- Submit abstracts with PI approval
- Practice presentations in lab meeting before the conference
- Prioritize clear visuals over text-dense slides following our Show, Don't Tell[3] principle
- Funding for conferences depends on availability and should be discussed in advance

22.4.1 Slides, posters, and figures

Our guidelines for slide decks, our presentation and poster resources, and our checklist for scientific figures have moved to Slides, posters, and figures[4] in *Principles of Scientific Writing*[5].

22.5 Grant Writing

- Graduate students and postdocs are encouraged to apply for fellowships (e.g., NIH F31, NSF GRFP, K awards)
- PIs will support fellowship applications with feedback, letters of support, and mentoring
- Start planning fellowship applications well in advance of deadlines (typically 3-6 months)
- Attend grant writing workshops and seek feedback from multiple sources

Resources:

- Building Your Funding Train[6]
- NIH Funding Opportunities[7]
- NIH Grant Writing Resources[8]

22.6 PhD Dissertation Requirements

Understanding what constitutes a sufficient PhD dissertation is crucial for setting realistic expectations and timelines. The dissertation represents an important milestone, but it doesn't need to be your magnum opus.

[3] @sec-show-dont-tell
[4] https://morrison-lab.github.io/psw/chapters/slides-posters-figures.html
[5] https://morrison-lab.github.io/psw/
[6] https://grantwriting.stanford.edu/resources/funding-train/
[7] https://grants.nih.gov
[8] https://www.niaid.nih.gov/grants-contracts/write-grant-application

22.6.1 Review Previous Dissertations

Before setting your dissertation goals, read previous dissertations from students in your program. This helps you:

- Understand the typical scope and depth expected
- See different approaches to structure and presentation
- Calibrate your expectations based on successful examples
- Identify common patterns and standards in your field

Most universities maintain electronic dissertation repositories, making it easy to access recent examples from your program.

22.6.2 Publication Requirements

Three first-author papers typically suffice for a dissertation in public health and biomedical sciences. If academic peers in reputable journals have approved your work through peer review, this demonstrates that your research meets professional standards. Your dissertation committee should recognize this external validation.

The specific publication requirements may vary by program and institution, so consult your program's guidelines and discuss expectations with your committee early. However, three substantial first-author publications generally demonstrate:

- Independent research capability
- Ability to communicate findings effectively
- Contribution to the scientific literature
- Readiness for an academic or research career

22.6.3 External Validation and Fast-Tracking

If you have a job offer waiting, you can usually get fast-tracked through the dissertation process. The reasoning is straightforward: your work has been externally validated as worthwhile by prospective employers.

Since most post-PhD positions offer better compensation than graduate stipends, it's difficult to justify prolonging your graduation when you've already demonstrated professional competence. This applies whether the job offer is in academia, industry, government, or nonprofit sectors.

22.6.4 Historical Context: The Masterpiece Tradition

The dissertation[9] (or thesis), a requirement for earning a PhD[10], is a spiritual successor to an apprentice's masterpiece[11] in craft guilds. Historically, a masterpiece was the piece of work that demonstrated an apprentice had achieved sufficient skill to join the guild as a master craftsperson. It was not meant to be the best work they would ever produce—it was meant to prove they were ready to work independently.

[9] https://en.wikipedia.org/wiki/Thesis
[10] https://en.wikipedia.org/wiki/Doctor_of_Philosophy#History
[11] https://en.wikipedia.org/wiki/Masterpiece#History

Similarly, your dissertation should demonstrate that you're ready to conduct independent research. It's your first professional-level work, not your career highlight. This perspective helps set appropriate expectations:

- The dissertation proves you can conduct rigorous research
- It doesn't need to solve every problem in your field
- It doesn't need to be flawless
- It doesn't need to be all-encompassing
- It just needs to constitute incremental progress in your field

22.6.5 Setting Realistic Expectations

Many PhD students struggle with perfectionism or "scope creep" in their dissertations. Remember:

- Done is better than perfect when it comes to dissertations
- You'll have your entire career to refine and expand on these ideas
- The goal is to finish and move forward, not to write the definitive work on your topic
- Your committee wants to see you succeed and graduate

Focus on making a solid, incremental contribution to knowledge in your field. That's what a dissertation is meant to be—no more, no less.

22.6.6 Resources

Dissertation writing tools:

- quarto-thesis[12] - Quarto extension for creating masters or PhD theses with professional LaTeX formatting

22.7 Teaching and Outreach

Teaching and outreach are valuable professional development opportunities:

- Graduate students are encouraged to gain teaching experience
- We support science communication and outreach activities
- Discuss opportunities with PIs

The UC Davis DataLab[13] offers various workshops and learning materials that can support your teaching and professional development. Their workshop index[14] provides a comprehensive catalog of available resources.

[12] https://github.com/nmfs-opensci/quarto-thesis
[13] https://github.com/ucdavisdatalab
[14] https://ucdavisdatalab.github.io/workshop_index/

22.8 Networking

Building a professional network is important for your career:

- Attend seminars and departmental events
- Connect with researchers in your field
- Join professional societies
- Use professional social media platforms to share research and engage with the scientific community

23 Writing

23.1 Writing to Clarify Your Thinking

Writing is a powerful tool for clarifying your thoughts, even before you start searching for answers. When questions or ideas come up, it's good practice to write them down immediately—this helps you:

- Organize your thoughts and identify what you actually need to know
- Articulate the problem more precisely
- Often realize the answer yourself through the process of writing

As Leslie Lamport, a Turing Award winner and computer scientist, states: "If you think you understand something, and don't write down your ideas, you only think you're thinking" (Lamport 2019).

23.2 Plain, Direct Prose

Write user-facing prose in a plain, direct style. This applies to everything a reader sees — pull request, issue, and commit text, docs, READMEs, code comments, release notes, and messages.

The guide of record is the lab's **Principles of Scientific Writing (PSW)**: https://morrison-lab.github.io/psw/. The rules below operationalize it. When PSW and this guidance disagree, PSW wins.

- **Limit dependent (subordinate) clauses.** One per sentence is plenty. When two or more stack up, split the sentence.
- **Cut low-content filler and jargon.** Delete words that add no information ("it is worth noting", "in order to" -> "to", "due to the fact that" -> "because").
- **Prefer plain (Anglish) words over Latin-derived ones** (PSW, "Word choice"). "before", not "prior to"; "needed", not "necessary"; "use", not "utilize". A heuristic, not a purity rule.
- **Prefer simple declarative sentences and active voice.** Subject, verb, point. Name the actor, then the action.
- **Join independent clauses with coordinating conjunctions** (and, but, so, or) over subordinate constructions. Prefer "X is fast, but Y is correct" over "While X is fast, Y is correct."

This is a default, not an absolute rule. Keep a clause or a technical term when removing it would lose meaning or precision. Never trade an honest hedge for false confidence.

23.3 Scanning for AI Tells

When AI tools draft or edit prose, the result often carries telltale signs of machine authorship.

Write plainly up front, then scan the draft for AI tells before sending. Before presenting non-trivial prose — pull request and issue descriptions, commit bodies, README, doc, and vignette text, or long answers — self-check it and cut the tells. Apply the same catalog when reviewing someone else's prose too:

- **Overused vocabulary:** actionable (-> "to act on"), delve, leverage, utilize, tapestry, testament, realm, robust, seamless, holistic, nuanced, multifaceted, pivotal, crucial, "in today's fast-paced world", "stands as a testament to".
- **Rhetorical reflexes:** the "it's not just X, it's Y" antithesis (the biggest tell), mechanical rule-of-three lists, signposting filler ("it is worth noting that", "importantly"), hedging stacks, hollow "in conclusion" restatements.
- **Structural and typographic:** em-dash overuse as a default connector, bold-leading `**Term:**` bullets applied mechanically, emoji section headers, conspicuously uniform paragraph rhythm.
- **Tonal:** promotional register, reflexive both-sidesing, vague universals with no concrete names or numbers.

De-slop — cut the filler and the reflexes — but do not ban words outright or sand the text into a flat, voiceless register. Any single tell is innocent; clustering and mechanical repetition are the signal. Code, terse status lines, and short conversational replies are exempt.

23.4 Show, Don't Tell

The adage "show, don't tell" (or "a picture is worth a thousand words") is a foundational communication principle across research manuscripts, technical documentation, and oral presentations.

Whenever conveying complex concepts, workflows, data patterns, or system architectures, favor concrete visual evidence and direct demonstration over dense verbal exposition.

23.4.1 In scientific manuscripts and reports

- **Lead with clear figures and tables:** Instead of describing multi-step analytical pipelines or multi-group comparative trends in lengthy paragraphs, present the data in a well-labeled plot or diagram.
- **Support visuals with interpretation, not repetition:** Use the narrative text to guide the reader through the key insights and takeaways of a figure rather than exhaustively listing every number already legible in a table or plot.
- **Use diagrams for conceptual models:** Directed acyclic graphs (DAGs), CONSORT flow diagrams, and workflow schematics clarify methodological structures much faster than textual descriptions alone.

23.4.2 In conference and lab presentations

- **Minimize text on slides:** Audiences cannot read dense bulleted paragraphs while simultaneously listening to a speaker. Replace text-heavy slides with clean visuals, diagrams, or summary charts.
- **Highlight key focal points:** Use callouts, annotations, or sequential visual reveals to focus attention on the specific relationship being discussed.
- **Demonstrate rather than assert:** When presenting software tools or analytical pipelines, show live output, reproducible code snippets, or screenshots rather than merely listing features.

24 Manuscript Preparation and Publication

24.1 Publication Process

The typical workflow for manuscript preparation and publication:

1. **Planning**: Discuss target journals, outline, and timeline with PIs
2. **Drafting**: Lead author prepares initial draft
3. **Internal review**: Co-authors review and provide feedback
4. **Revision**: Lead author incorporates feedback iteratively
5. **PI approval**: Obtain final approval from PIs before submission
6. **Submission**: Submit to journal
7. **Revisions**: Lead author coordinates response to peer reviewers
8. **Publication**: Celebrate and share!

24.2 Responding to Peer Review

When a journal asks for revisions, you will typically need to provide a point-by-point response to the reviewers' comments. The key strategy is to use the "yes-and" approach[1] from improvisational theater—building on what reviewers say rather than contradicting them. Try to avoid explicitly disagreeing with reviewers if at all possible.

Standard response format:

The typical response follows this pattern:

> "Thank you for raising this important point. We agree, and now state (page ##):
>
> [quote a section of the revised manuscript here]"

Balancing direct response and manuscript content:

There can be some overlap between the direct response to the reviewer (the sentences before "now state") and what goes into the revised manuscript (the part after "(page ##)"). This is acceptable, but we usually try to put as much of the content into the revised manuscript as possible, and minimize the direct response section. The goal is to make the manuscript self-contained while showing the reviewer that you've addressed their concern.

When content already exists:

Sometimes, a reviewer asks for content that was already in the manuscript. In these cases, first see if there's anything you can elaborate or clarify. Even if you don't see room for improvement, you can usually respond with:

[1] https://en.wikipedia.org/wiki/Yes,_and_...

> "We agree, and now state (page ##):
>
> [quote the content that was already there before]"

The reviewer doesn't need to know that they missed it the last time (see also: implicature[2].

Additional tips:

- Always thank reviewers for their time and feedback
- Quote page numbers from the revised manuscript
- Use direct quotes from your revised text when possible
- Maintain a respectful and collaborative tone throughout
- If you make changes beyond what the reviewer requested, mention them briefly

24.3 Preprints and Open Access

- We encourage posting preprints prior to or during peer review on platforms like medRxiv[3] or bioRxiv[4]
- Preprints allow rapid dissemination of findings and can be cited in grant applications
- We support publishing in open access journals when possible to maximize accessibility
- Many funders, including NIH, have public access policies that require making publications freely available

A preprint is a scientific manuscript that has not been peer reviewed. Preprint servers create digital object identifiers (DOIs) and can be cited in other articles and in grant applications. Because the peer review process can take many months, publishing preprints prior to or during peer review enables other scientists to immediately learn from and build on your work. Importantly, NIH allows applicants to include preprint citations in their biosketches. In most cases, we publish preprints on medRxiv[5].

24.4 Reporting Checklists

Using reporting checklists ensures that publications contain information needed for readers to assess validity and reproducibility. We use checklists appropriate to study design:

- CONSORT for randomized trials
- STROBE for observational studies
- PRISMA for systematic reviews
- Others as appropriate (see EQUATOR Network[6])

[2] https://en.wikipedia.org/wiki/Implicature
[3] https://www.medrxiv.org/
[4] https://www.biorxiv.org/
[5] https://www.medrxiv.org/
[6] https://www.equator-network.org/

24.5 Manuscript Checklist

Before submitting a manuscript:

- ☐ Completed relevant reporting checklist
- ☐ Results are reproducible (rerunning code replicates tables/figures exactly)
- ☐ Target journal selected
- ☐ Title is declarative and states findings clearly
- ☐ Word count meets journal requirements
- ☐ Manuscript follows journal formatting guidelines
- ☐ Consistent voice throughout (typically active voice)
- ☐ All figures and tables referenced in main text
- ☐ Captions for all figures and tables
- ☐ Tables/figures numbered by order of appearance
- ☐ Abbreviations defined at first use and used sparingly
- ☐ Avoid subjective wording (e.g., "interesting", "dramatic")
- ☐ Avoid directionless statements (specify direction of associations)
- ☐ Causal language only when supported by study design
- ☐ Avoid "significant" (easily confused with statistical significance)
- ☐ Author contributions drafted using CRediT Taxonomy

24.6 Formatting tables with `gtsummary` + `flextable` + `flexlsx`

One practical workflow for publication-ready tables is to use `{gtsummary}`[7] to create summary/regression tables, convert to `{flextable}`[8] for styling, and export to Excel with `{flexlsx}`[9].

````
```{r}
#| eval: false

tbl1 <-
  trial |>
  gtsummary::tbl_summary(by = trt) |>
  gtsummary::as_flex_table() |>
  flextable::autofit()

flexlsx::write_xlsx(tbl1, path = here::here("results", "table-1.xlsx"))
```
````

This gives you a reproducible table pipeline:

- Define table content with `gtsummary::tbl_summary()` (or `gtsummary::tbl_regression()` for model tables).
- Apply presentation formatting with `flextable` verbs.
- Export an `.xlsx` file for collaborator review while preserving your R workflow.

[7] https://www.danieldsjoberg.com/gtsummary/
[8] https://davidgohel.github.io/flextable/
[9] https://cran.r-project.org/package=flexlsx

24.7 Scientific Writing: Claims and Evidence

All factual claims in scientific writing should be supported by appropriate evidence.

Citation requirements:

- **Cite primary sources** for factual statements about established knowledge, methods, or findings
- **Cite official documentation** when describing how software, tools, or systems work
- **Link to authoritative sources** like peer-reviewed publications, official repositories, or technical specifications
- Avoid citing secondary sources when primary sources are available

When you can't find a citation:

- **Demonstrate directly**: Show the behavior through experiments, data, or explicit examples
- **Acknowledge uncertainty**: Use appropriate hedging language ("may", "appears to", "in our experience") when evidence is limited
- **Remove the claim**: If you cannot substantiate a claim with either citations or direct evidence, consider whether it needs to be included

Why this matters:

- Builds reader trust and credibility
- Enables readers to verify information independently
- Maintains scientific rigor in all communications
- Prevents propagation of misinformation

This principle applies to all lab writing, including: manuscripts, documentation, grant applications, and technical reports.

Using AI tools for writing:

When using AI tools to help develop manuscripts or other academic writing, follow the special guidance in the wai site's "Using AI for Journal Articles" section[10] to ensure transparency, maintain intellectual ownership, and avoid plagiarism.

[10] https://morrison-lab.github.io/wai/chapters/ai-use-policies.html#sec-ai-journal-articles

References

Benjamin-Chung, Jade, Kunal Mishra, Stephanie Djajadi, et al. 2024. *Benjamin-Chung Lab Manual*. https://jadebc.github.io/lab-manual/.

Crameri, Fabio, Grace E. Shephard, and Philip J. Heron. 2020. "The Misuse of Colour in Science Communication." *Nature Communications* 11. https://doi.org/10.1038/s41467-020-19160-7.

"Creative Commons Attribution-NonCommercial 4.0 International License." n.d. Creative Commons. http://creativecommons.org/licenses/by-nc/4.0/.

"CRediT: Contributor Roles Taxonomy." n.d. PLOS ONE. https://journals.plos.org/plosone/s/authorship.

"Dryad Digital Repository." n.d. Dryad. https://datadryad.org/.

"EQUATOR Network: Enhancing the QUAlity and Transparency of Health Research." n.d. EQUATOR Network. https://www.equator-network.org/.

"How to Store and Manage Your Data." n.d. PLOS. https://plos.org/resource/how-to-store-and-manage-your-data/.

International Committee of Medical Journal Editors. n.d. "Defining the Role of Authors and Contributors." ICMJE. http://www.icmje.org/recommendations/browse/roles-and-responsibilities/defining-the-role-of-authors-and-contributors.html.

Lamport, Leslie. 2019. *Think and Write with Leslie Lamport*. Podcast interview. https://mentors.fm/2019/08/13/think-and-write-with-leslie-lamport/.

"medRxiv: The Preprint Server for Health Sciences." n.d. Cold Spring Harbor Laboratory. https://www.medrxiv.org/.

Munafò, Marcus R., Brian A. Nosek, Dorothy V. M. Bishop, et al. 2017. "A Manifesto for Reproducible Science." *Nature Human Behaviour* 1. https://doi.org/10.1038/s41562-016-0021.

Nuzzo, Regina. 2015. "How Scientists Fool Themselves – and How They Can Stop." *Nature* 526: 182–85. https://doi.org/10.1038/526182a.

Ojiako, Chiamaka P., Lazenya Weekes-Richemond, Vuyiseka Dubula-Majola, and Marie-Claire Wangari. 2023. "Who Is a Global Health Expert?" *PLOS Global Public Health* 3 (8): e0002269. https://doi.org/10.1371/journal.pgph.0002269.

"Open Science Framework." n.d. Center for Open Science. https://osf.io/.

"Slurm Workload Manager: Salloc Documentation." n.d.-a. SchedMD. https://slurm.schedmd.com/salloc.html.

"Slurm Workload Manager: Sattach Documentation." n.d.-b. SchedMD. https://slurm.schedmd.com/sattach.html.

"Slurm Workload Manager: Sbatch Documentation." n.d.-c. SchedMD. https://slurm.schedmd.com/sbatch.html.

"Slurm Workload Manager: Squeue Documentation." n.d.-d. SchedMD. https://slurm.schedmd.com/squeue.html.

"Slurm Workload Manager: Srun Documentation." n.d.-e. SchedMD. https://slurm.schedmd.com/srun.html.

"Ssh_config(5) Manual Page." n.d. OpenBSD. https://man.openbsd.org/ssh_config.

"Ssh_config(5) Manual Page, Debian Edition." n.d. Debian Project. https://manpages.debian.org/stable/openssh-client/ssh_config.5.en.html.

Stoddart, Charlotte. 2019. "Is There a Reproducibility Crisis in Science?" *Nature*, ahead of print. https://doi.org/10.1038/d41586-019-00067-3.

White, Michael. 2019. "How Small Changes to a Paper Can Help to Smooth the Review Process." *Nature*, ahead of print. https://doi.org/10.1038/d41586-019-01431-z.

Copilot Instructions File

For the complete `.github/copilot-instructions.md` file, please view the HTML version of this appendix at: https://ucd-serg.github.io/lab-manual/appendix-copilot-instructions.html

Copilot Setup Steps File

For the complete `.github/workflows/copilot-setup-steps.yml` file, please view the HTML version of this appendix at: https://ucd-serg.github.io/lab-manual/appendix-copilot-setup-steps.html

Document Generation Metadata

This document was generated from the following git commit:

- **Branch:** HEAD
- **Commit:** 5093847
- **Full commit hash:** 509384717d63f033334f7dc7f3951c1cb89aa253
- **Commit date:** 2026-10-12 08:10:06 +0000

When transferring edits from this DOCX file back to the Quarto source files, use this commit information to set up the PR correctly and account for any commits that have been added since this document was generated.